

Transformers

cnrs

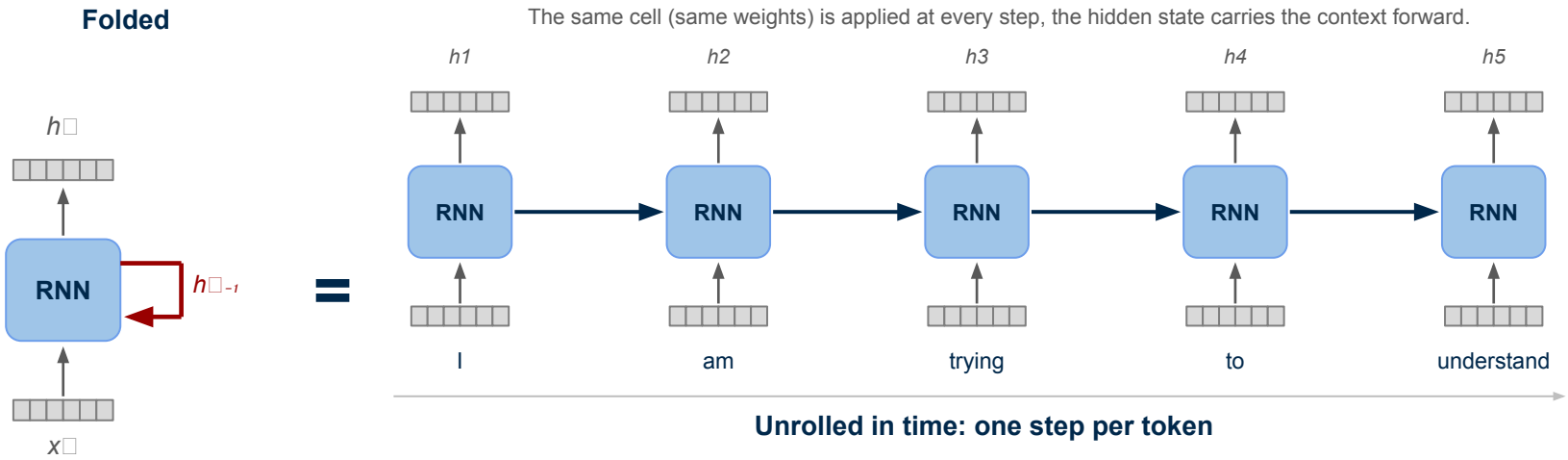
CC BY NC SA



From RNN to LLM

A long time ago: recurrent neural networks

Recurrent neural network (RNN) : reads a sequence one token at a time and updates a hidden state that summarizes everything read

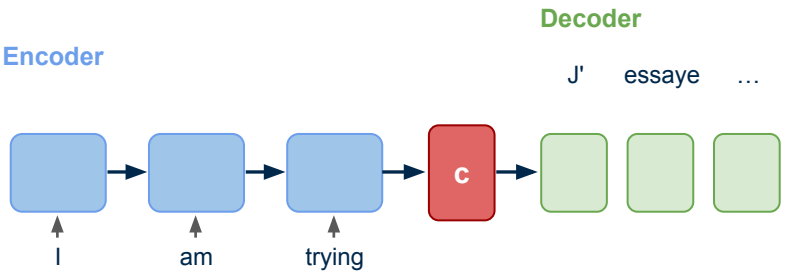


- **Sequential:** step t must wait for step t − 1, so training cannot be parallelized over the sequence.
- **Short memory:** information from distant tokens fades (vanishing gradients).

Seq2seq and the first attention (2014)

Attention : at each output step, the decoder takes a weighted average of all encoder states instead of relying on one fixed vector.

1 · The bottleneck: one fixed vector



Encoder–decoder RNN (Sutskever et al., 2014). The encoder squeezes the English sentence into one vector **c**.

The decoder must write the whole French sentence from **c** alone: quality drops on long sentences.

2 · Attention: the decoder looks back at every word

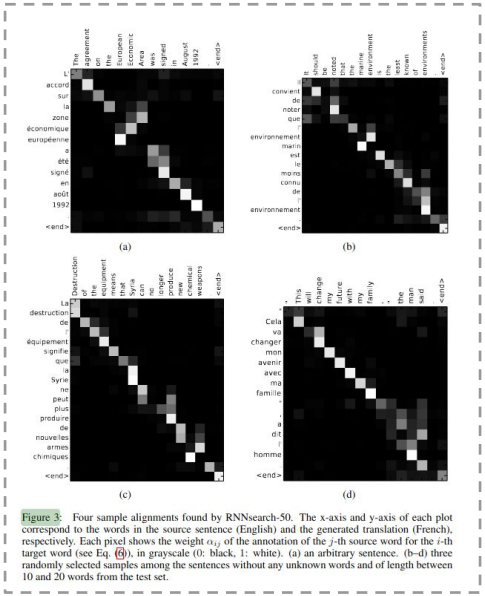


Figure 3: Four sample alignments found by RNNsearch-50. The x-axis and y-axis of each plot correspond to the words in the source sentence (English) and the generated translation (French), respectively. Each pixel shows the weight α_{ij} of the annotation of the j-th source word for the i-th target word (see Eq. (6)), in grayscale (0: black, 1: white). (a) an arbitrary sentence. (b–d) three randomly selected samples among the sentences without any unknown words and of length between 10 and 20 words from the test set.

What the paper found. Each square is the weight between an English word (column) and the French word being written (row): white = 1, black = 0.

Mostly diagonal: words are translated in order.

Off the diagonal: in (a), “European Economic Area” becomes “zone économique européenne”: the model looks at the words in reverse order.

No alignment labels: the weights are learned from translation alone.

Bahdanau et al. (2014), “Neural Machine Translation by Jointly Learning to Align and Translate”, Figure 3
https://arxiv.org/abs/1409.0473

2017: Attention Is All You Need

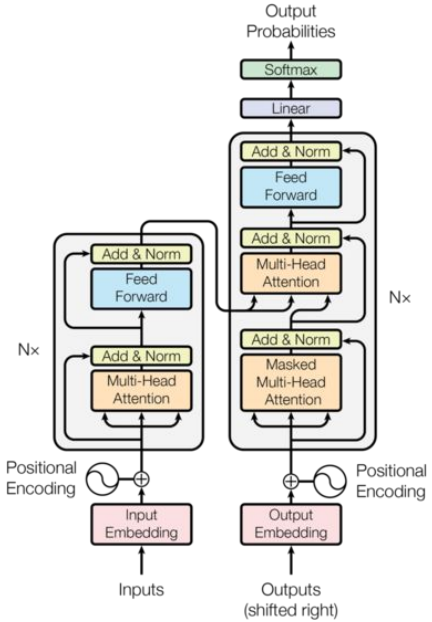
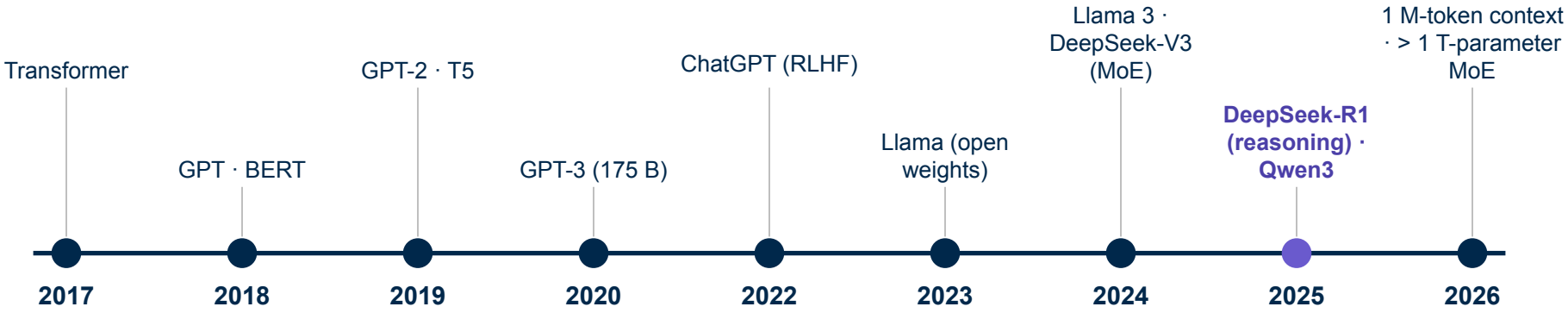


Figure 1 of Vaswani et al., NeurIPS 2017 (arXiv:1706.03762)

- **No recurrence:** all tokens are processed in parallel.
- Any token reaches any other in a single step.
-
- **Base model :** encoder–decoder, 6 + 6 layers, 65 M parameters (base model).

From the Transformer to today's LLMs

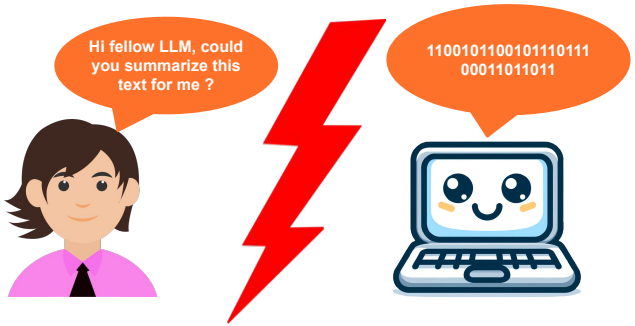
Large language model (LLM) a Transformer trained on trillions of tokens to predict the next token, then adapted to follow instructions.





Introduction

The problem of NLP: Sparse data



Traditional NLP (until the 2010s)

1 · Words split with hand-written rules → **2 · Normalize** lowercase, stop words, stems → **3 · Count** bag of words, TF-IDF → **4 · Classify** logistic regression, SVM

Example: “The cats are sleeping” → cat, sleep → one count per vocabulary word.

Documents × vocabulary: a huge, empty matrix

	aardvark	…	cat	…	dog	…	king	…	queen	…	sleep	…	zoo
doc 1	0		**1**		0		0		0		**1**		0
doc 2	0		0		**2**		0		0		0		0
doc 3	0		0		0		**1**		**1**		0		0
doc 4	0		**1**		0		0		0		0		0
doc 5	0		0		0		0		0		**3**		0

One-hot words

king	0	0	**1**	0	0	0	…
queen	0	0	0	**1**	0	0	…
mercenary	0	0	0	0	0	**1**	…

The main problems

- Huge and almost empty
- No meaning: king is as far from queen as from mercenary
- Word order is lost
- Unknown words become <UNK>

Tokenization

Tokenizer : splits text into tokens and maps each one to an integer ID. It is built once, before training the model.

Anti - *chocs*, **constitution**
et *actu* - ***elle*** - ***ment*** sont appris.

Anticonstitutionnellement sera compris.

Tokenizers

Model	Year	Algorithm	Vocabulary
BERT	2018	WordPiece	30,522
GPT-2	2019	Byte-level BPE	50,257
T5	2019	Unigram (SentencePiece)	32,000
Llama 2	2023	BPE (SentencePiece)	32,000
Llama 3	2024	Byte-level BPE	128,256
Qwen3	2025	Byte-level BPE	**151,669**
Gemma 3	2025	SentencePiece	≈ 262k

BPE (Byte-Pair Encoding)

BPE : start from characters and repeatedly merge the most frequent adjacent pair into a new token.

1. Training corpus

Word	Count	Start
low	5	l o w
lower	2	l o w e r
newest	6	n e w e s t
widest	3	w i d e s t

2. Merge the most frequent pair, repeat

Step	Merge	Count	New token
1	e + s	9	es
2	es + t	9	est
3	l + o	7	lo
4	lo + w	7	low
5	n + e	6	ne
6	ne + w	6	new

3. Encode new words

Apply the merges, in the same order:

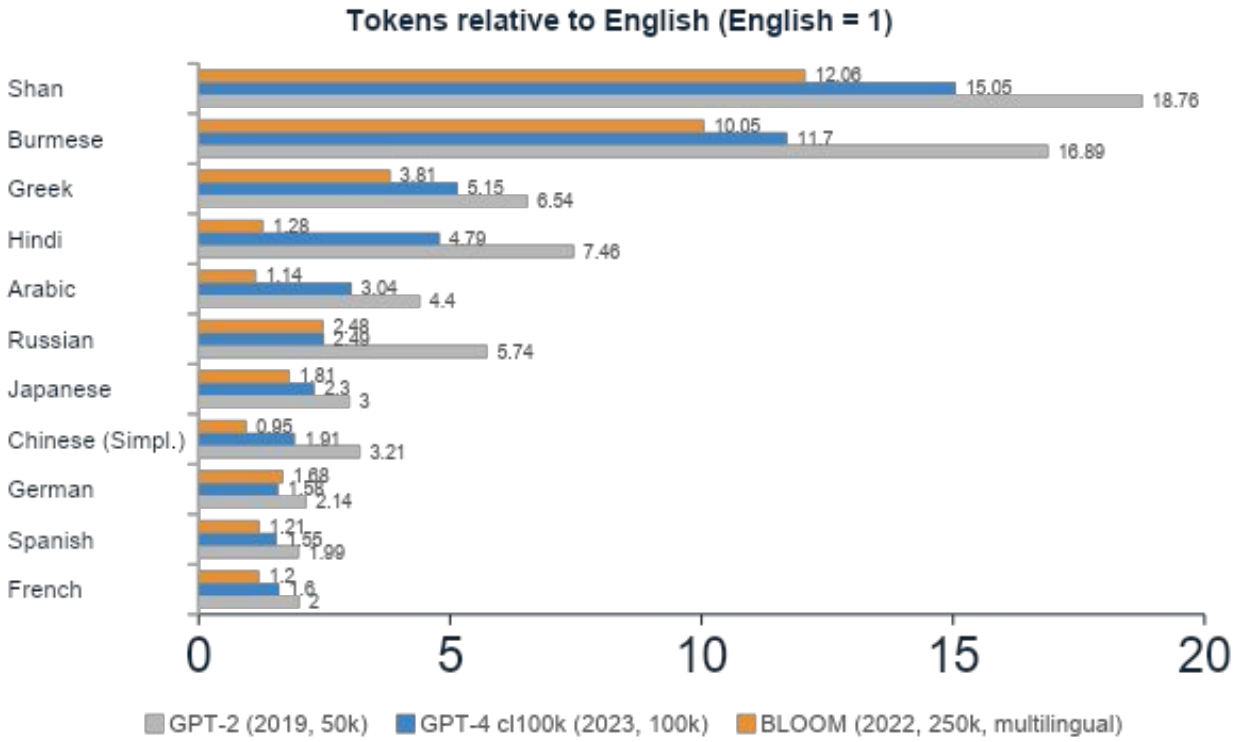
`lowest` → **low** **est**

`newer` → **new** **e** **r**

`slow` → **s** **low**

Sennrich et al., ACL 2016 (retrouver le lien)

Tokens are not equal across languages



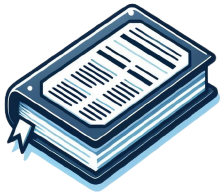
More tokens means more compute, a higher cost per request and less text in the context window.

Data: Petrov et al., NeurIPS 2023, FLORES-200 (github.com/AleksandarPetrov/tokenization-fairness, MIT).

https://arxiv.org/pdf/2305.15425

Embeddings

Embedding : a learned dense vector for each token. The embedding matrix E has one row per vocabulary token: V × d.

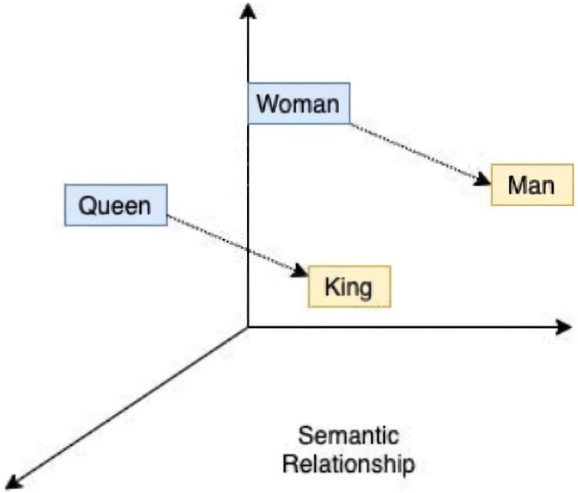


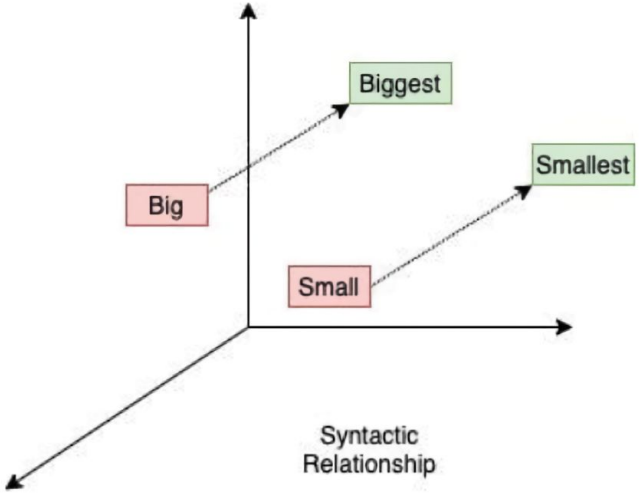
- king
- queen
- riots
- revolution
- table
- chair
- cable



- 0.2, 1.5, …
- 0.2, -1.5, …
- 1.3, -0.1, …
- -0.75, 0.05, …
- -0.3, 0.2, …
- 1.5, 0.5, …
- 0, 0, …

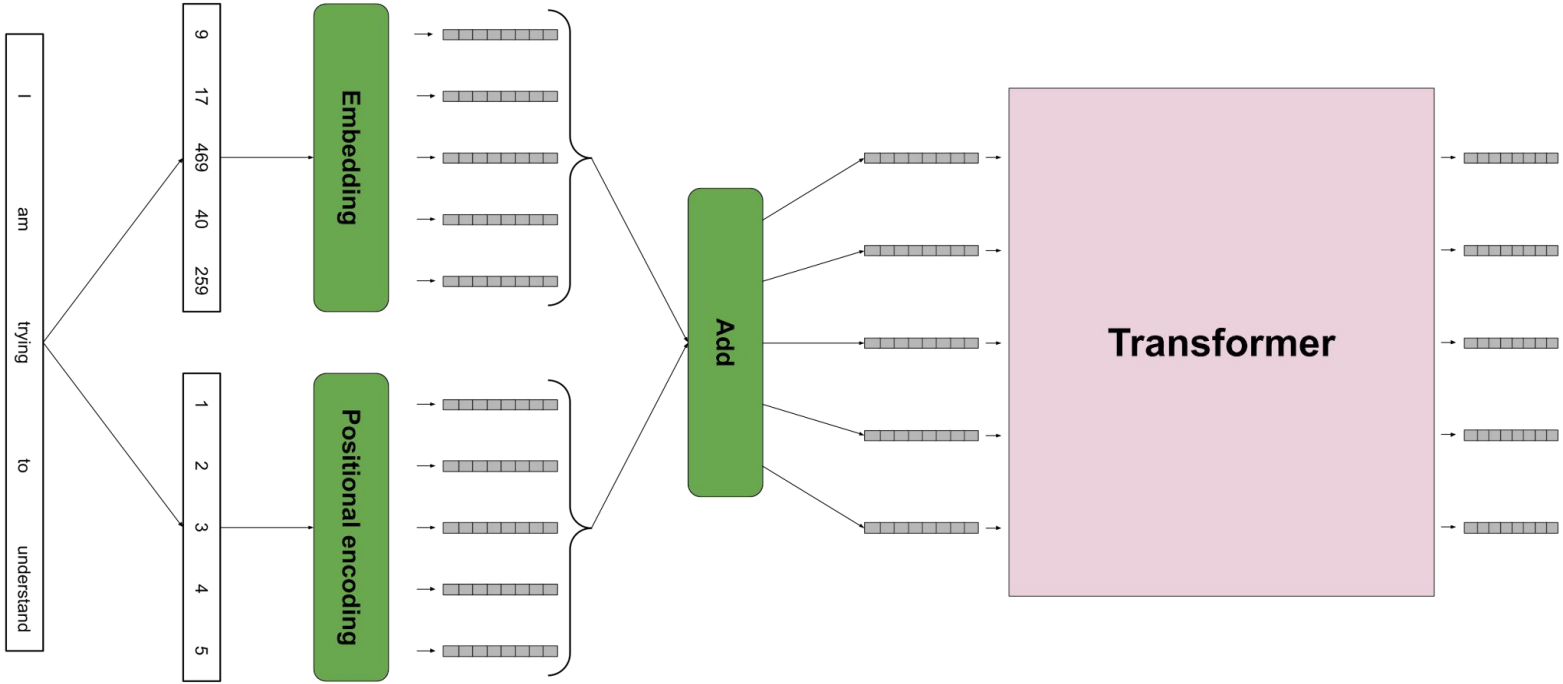
Embedding capture relationship between words





Pre-processing in NLP with Transformers

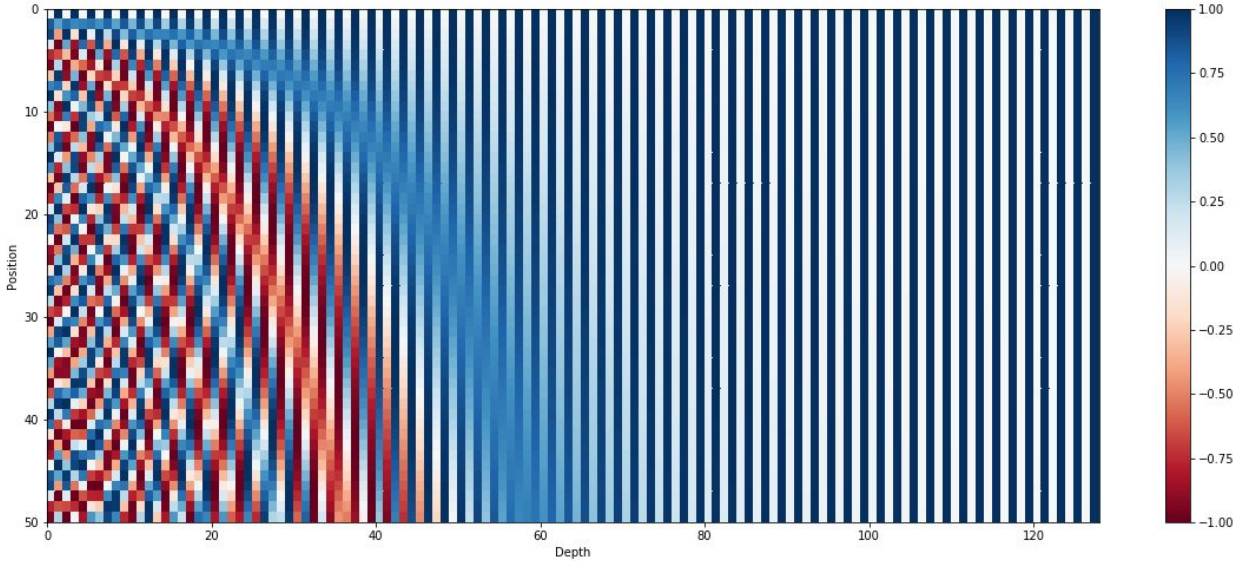
Model input : one vector per token, its embedding (what) and its positional encoding (where)



Positional encoding

Qwen3-8B no vector added at the input: RoPE inside attention

Sinusoidal positional encoding — a fixed vector of sines and cosines at different frequencies, unique to each position. *(Vaswani et al., 2017)*



$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

pos the position of the word in the sentence

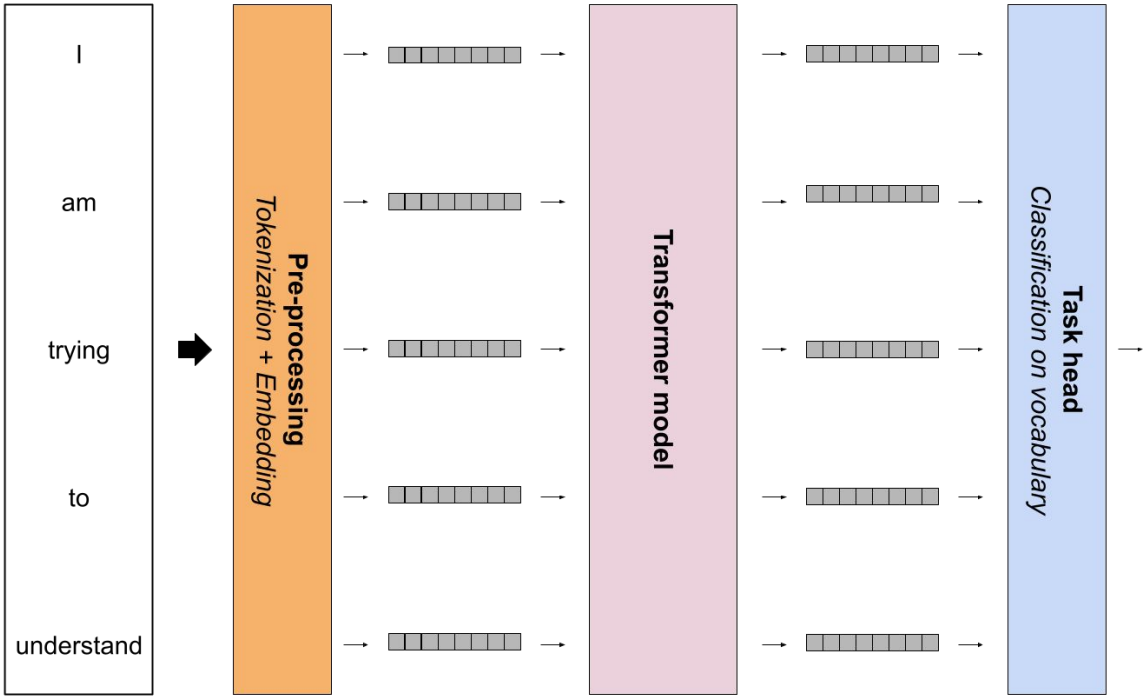
i the dimension (index of a coordinate in the embedding)

d_{model} the total dimension of the embedding

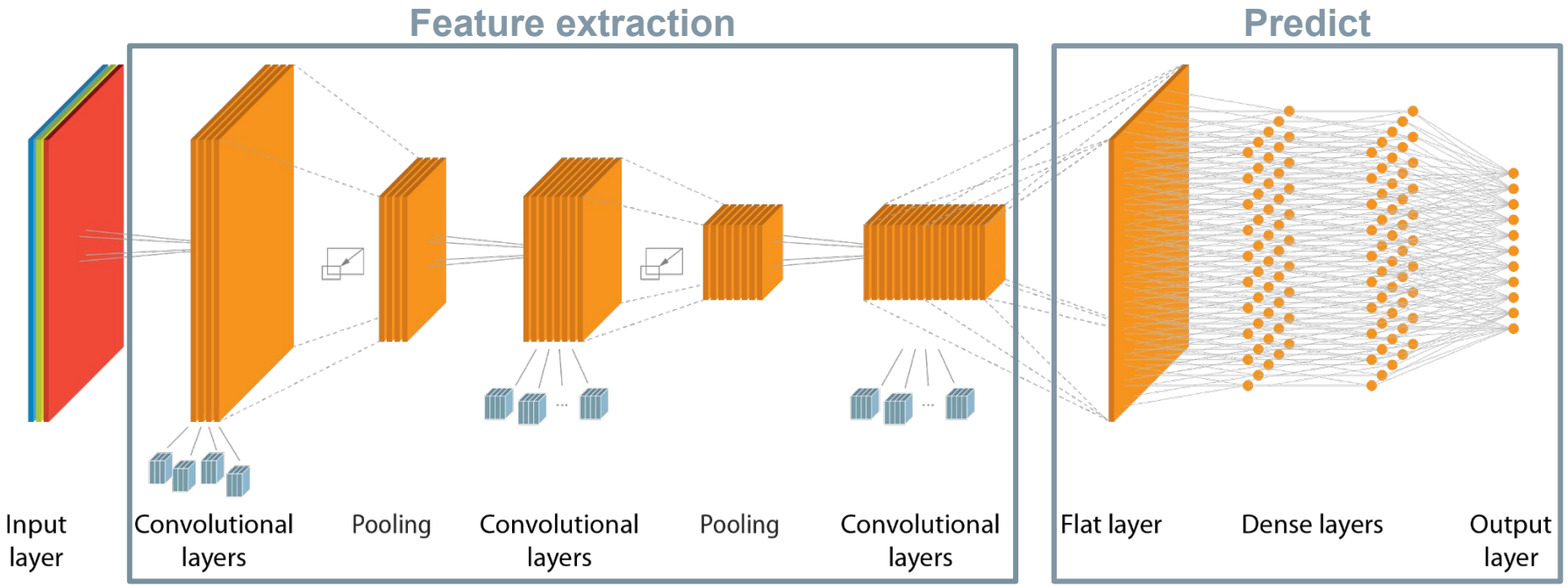


Transformers fundamentals

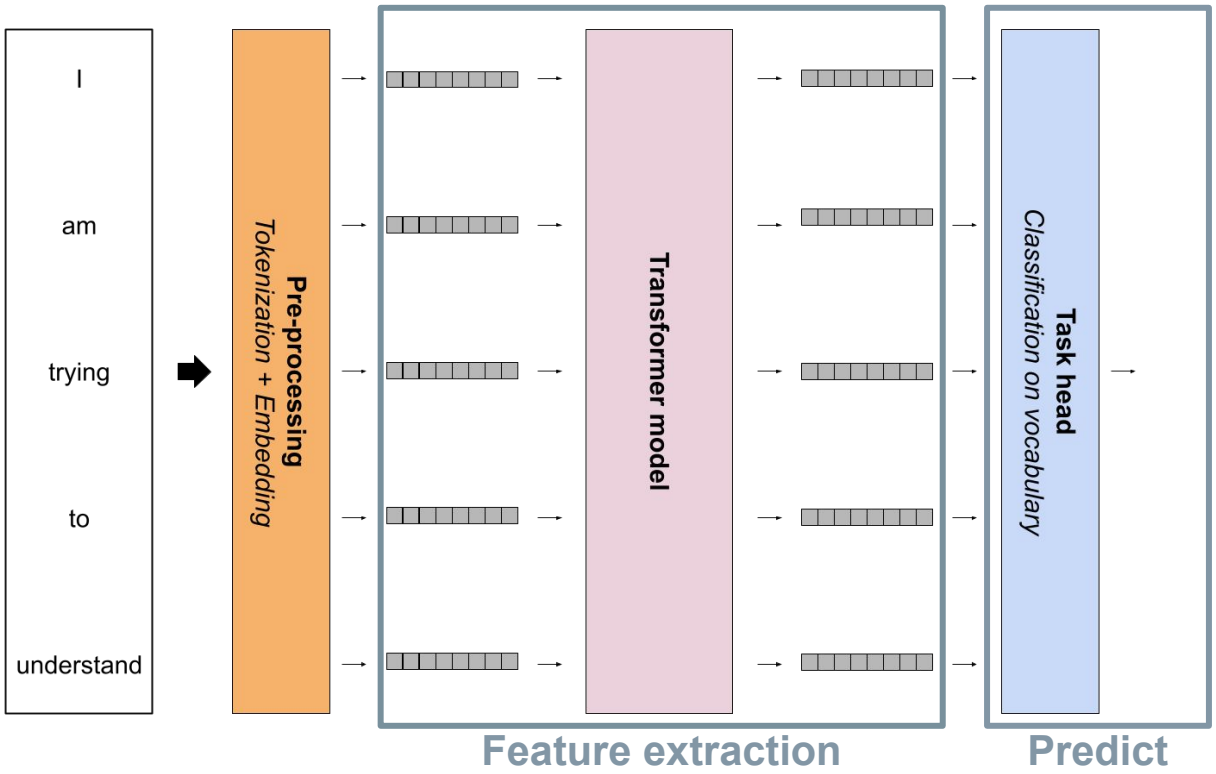
Example of NLP system



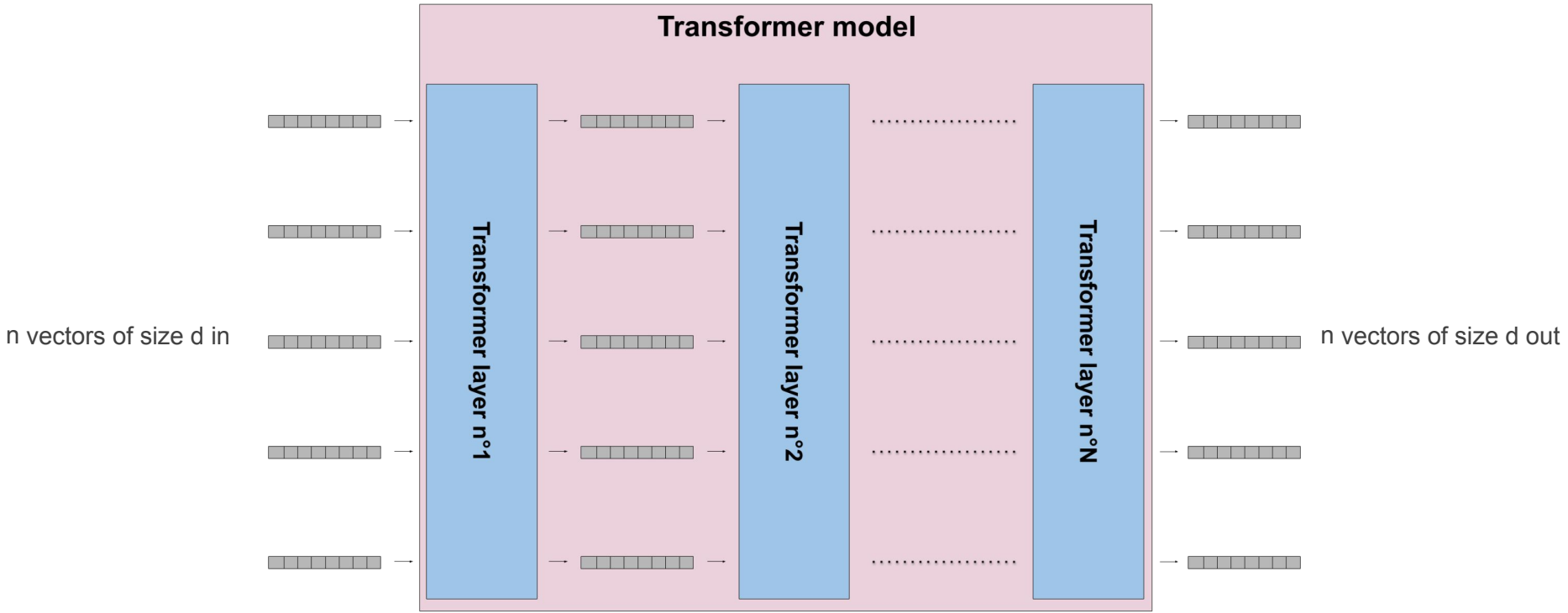
Analogy with vision: a CNN image classifier



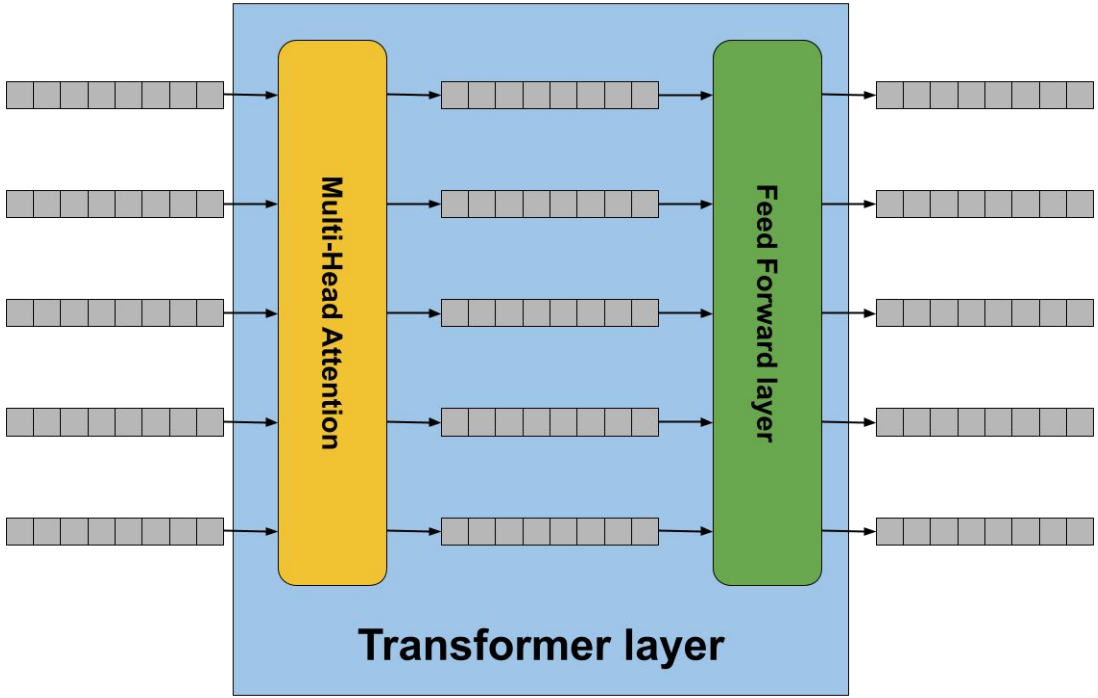
Feature extraction with Transformers



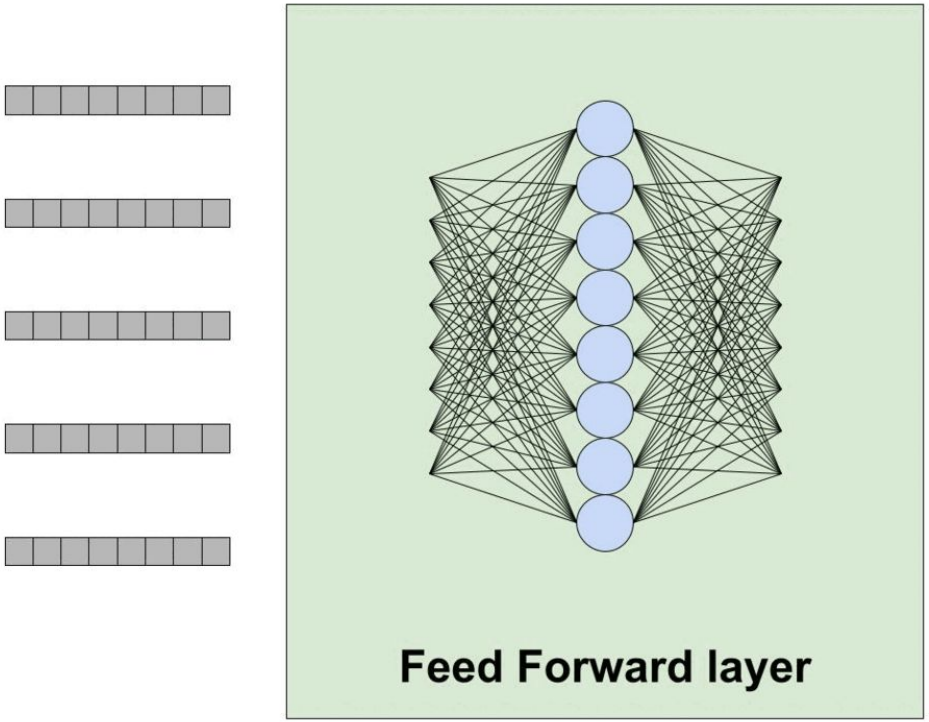
Vanilla Transformer architecture



Transformer layer simplified

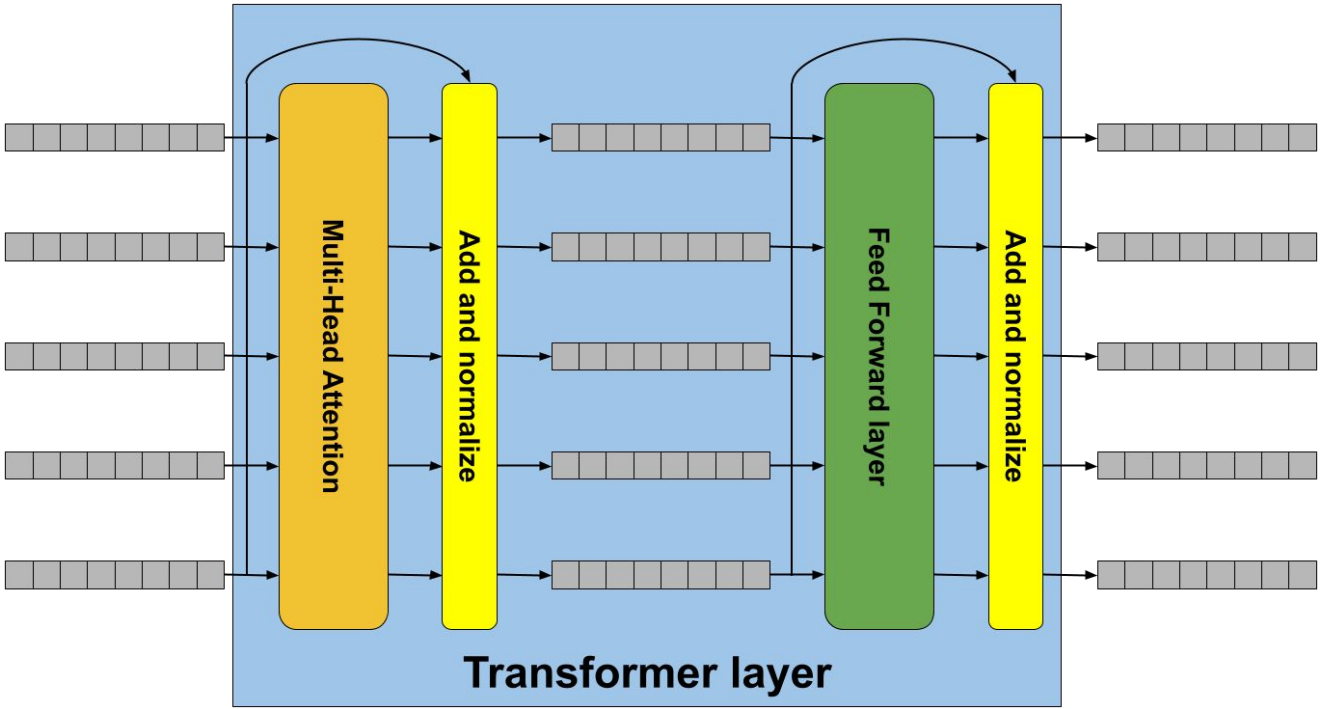


Feed Forward layer



Transformer layer

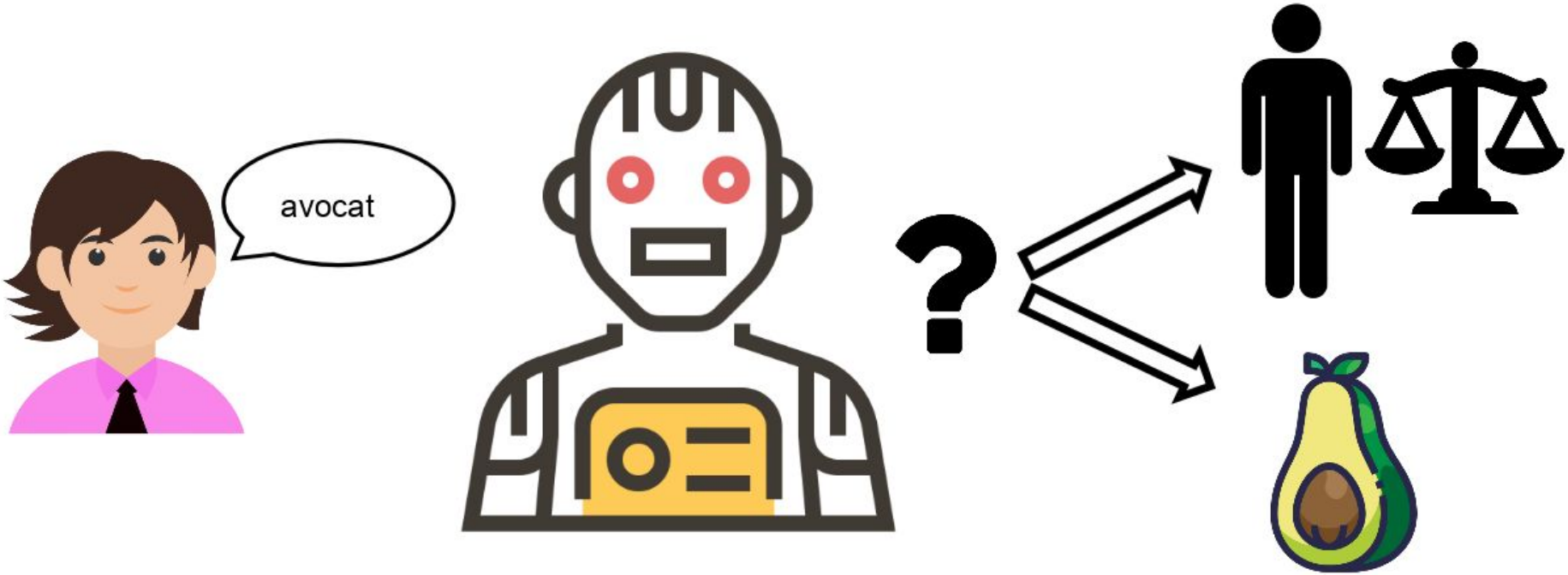
Residual connection : each sub-layer adds a correction to its input, a normalization then rescales the vector.





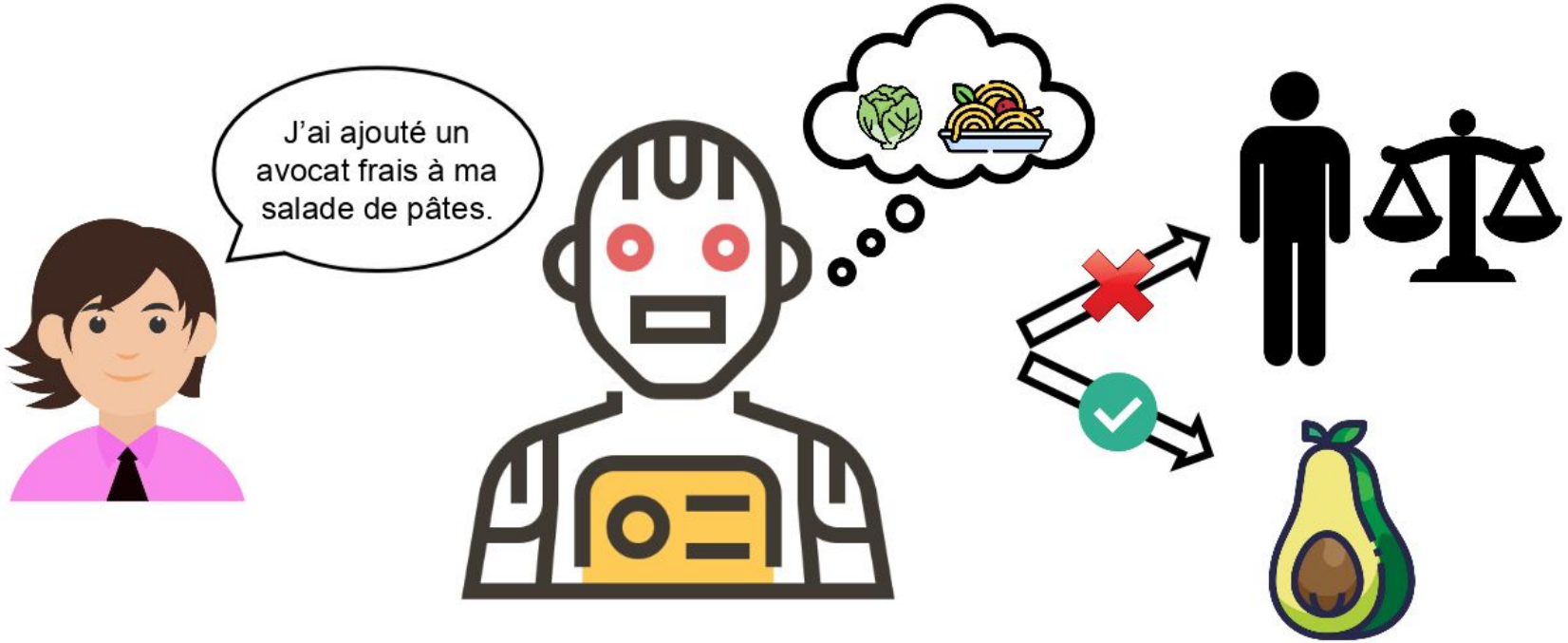
Attention mechanism

One word, several meanings

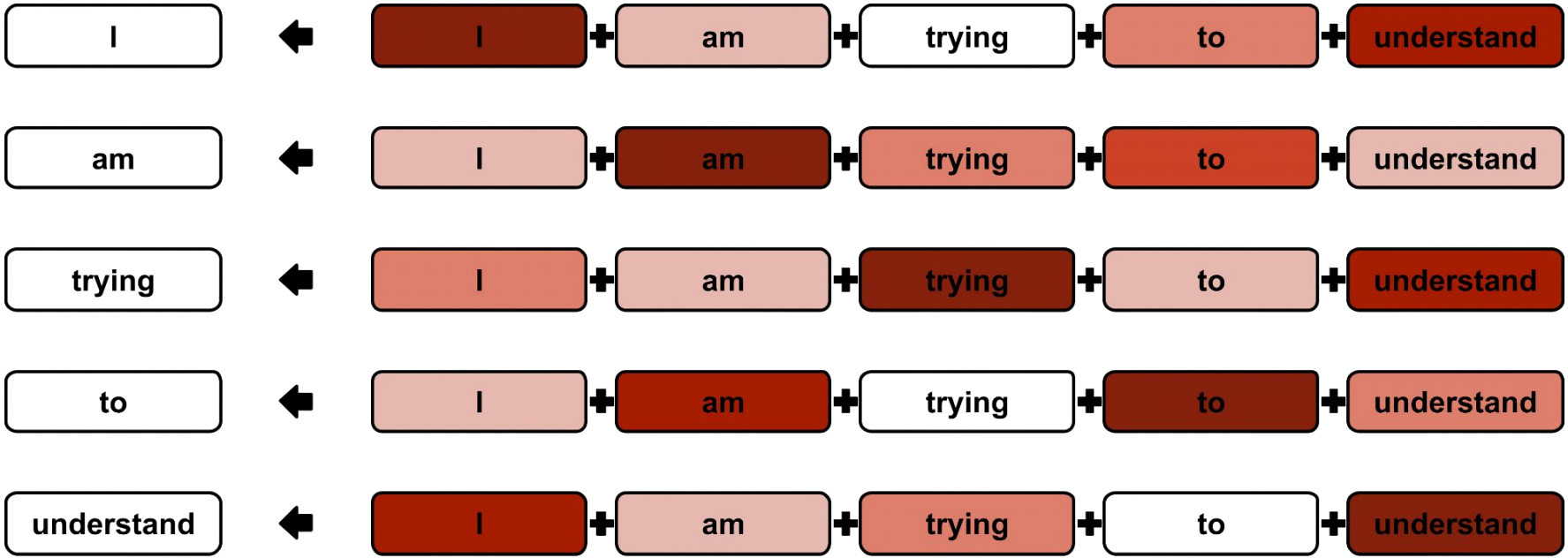


Context decides

Context : the surrounding words that tell which meaning is meant. Attention lets every token read its context.

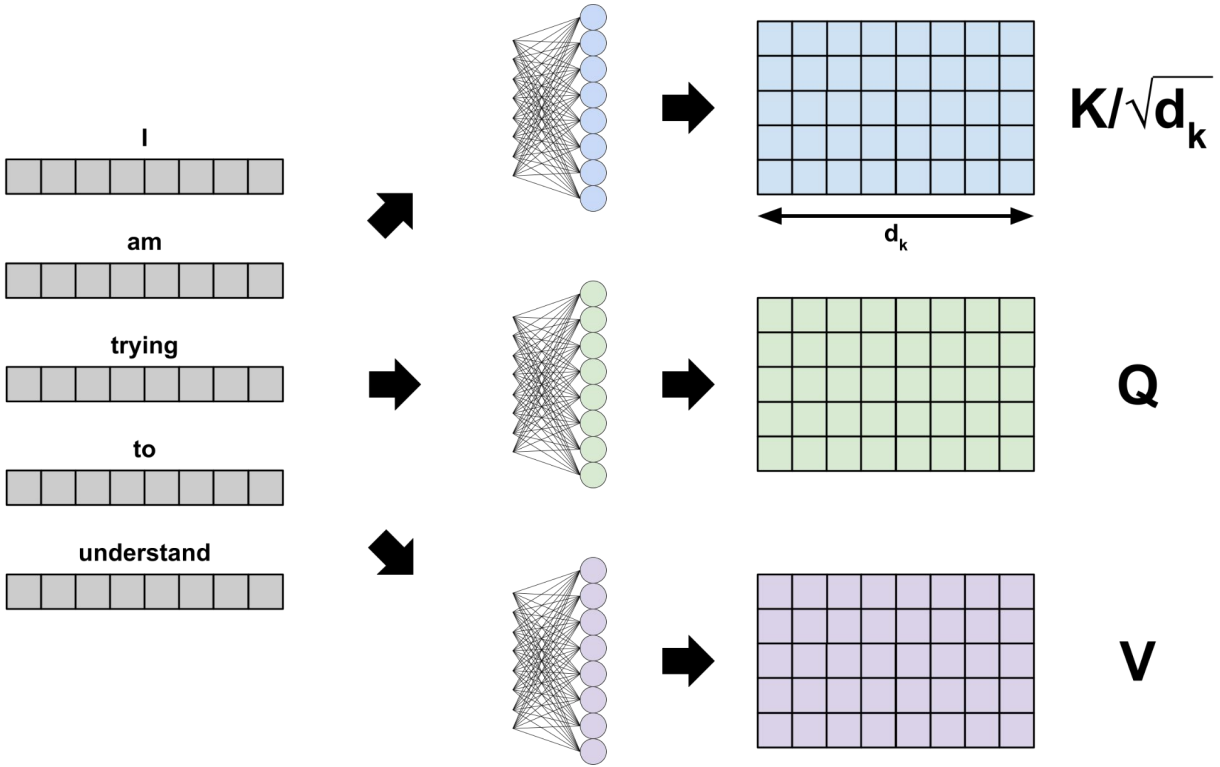


Intuition behind the attention mechanism



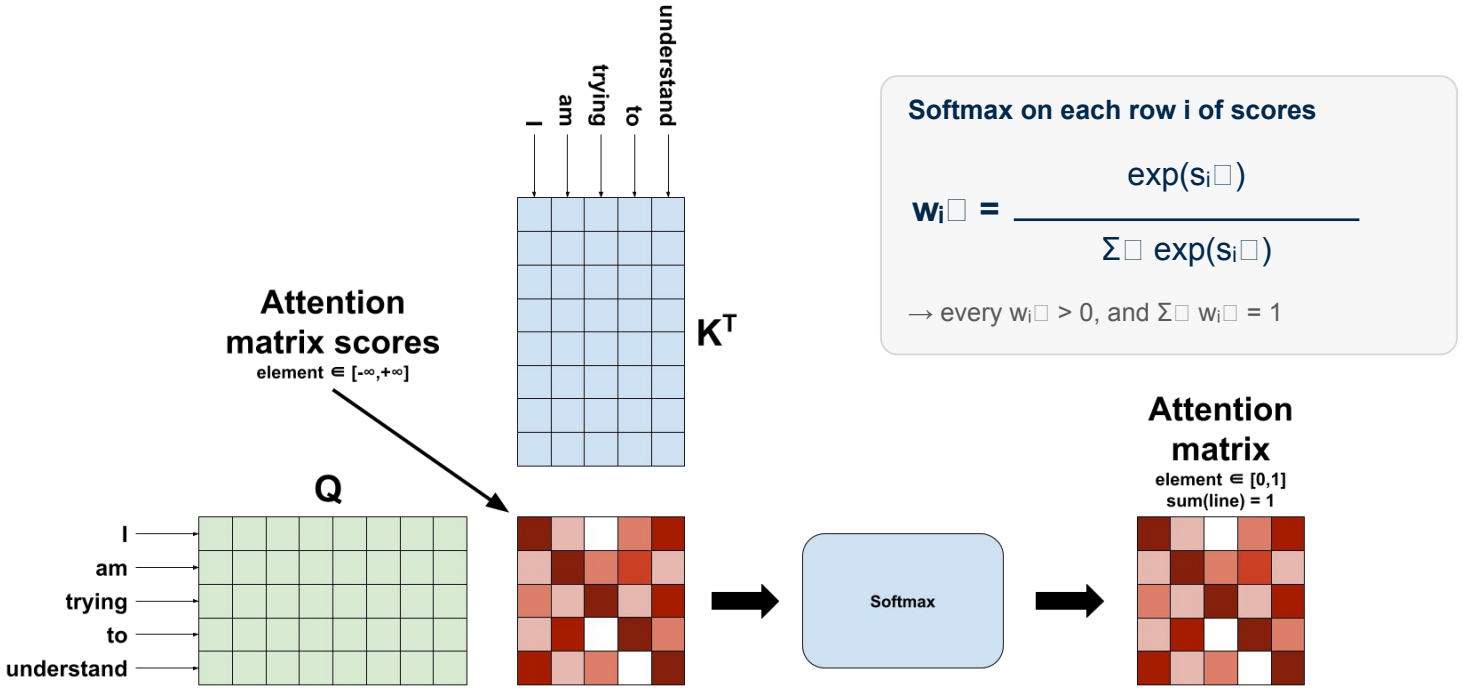
Attention explained - 1

Query, key, value : three learned projections of each token: what it looks for (Q), what it offers (K), what it passes on (V).



Attention explained - 2

Softmax : turns each row of scores into positive weights that sum to 1: the attention matrix.

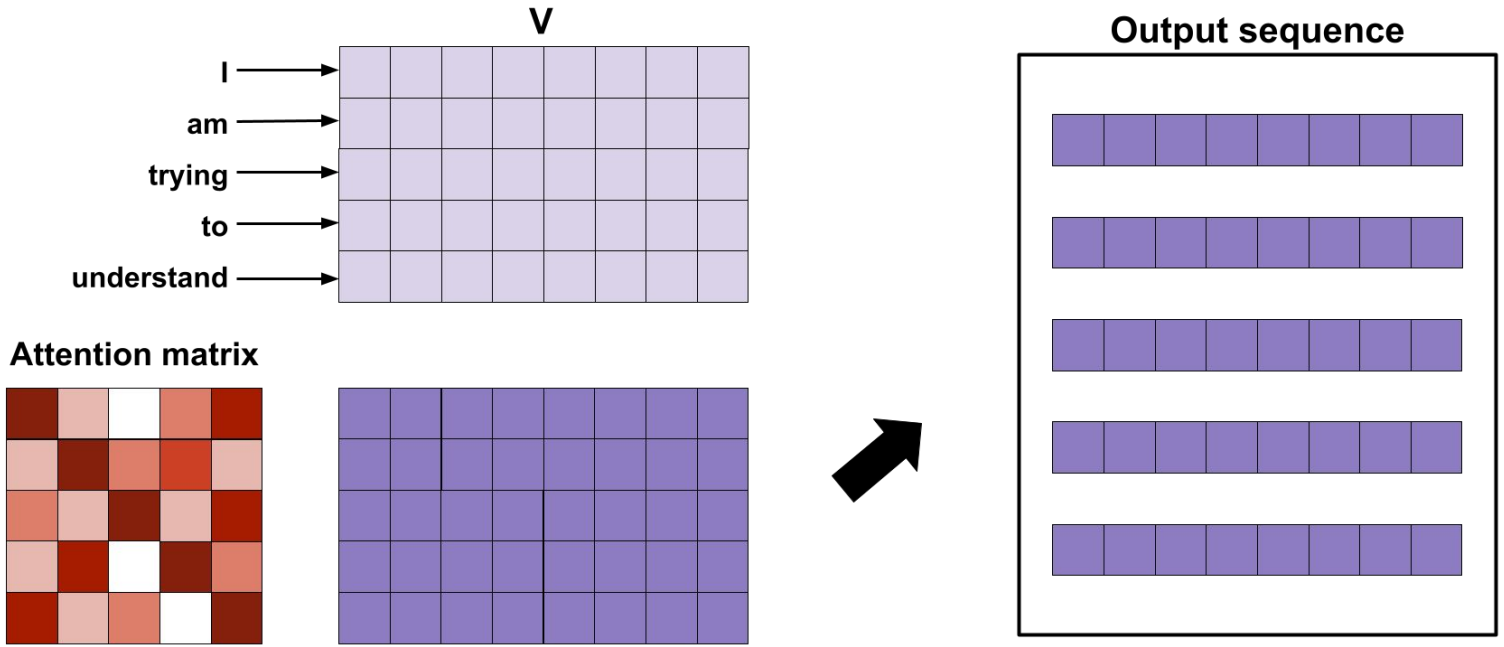


Softmax on each row i of scores

$$w_{i\square} = \frac{\exp(s_{i\square})}{\Sigma_{\square} \exp(s_{i\square})}$$

→ every $w_{i\square} > 0$, and $\Sigma_{\square} w_{i\square} = 1$

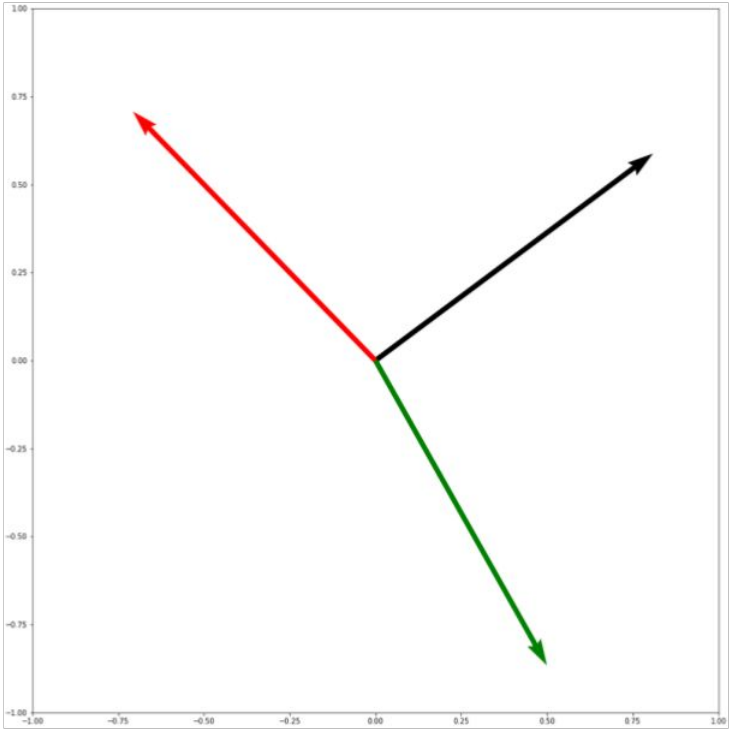
Attention explained - 3



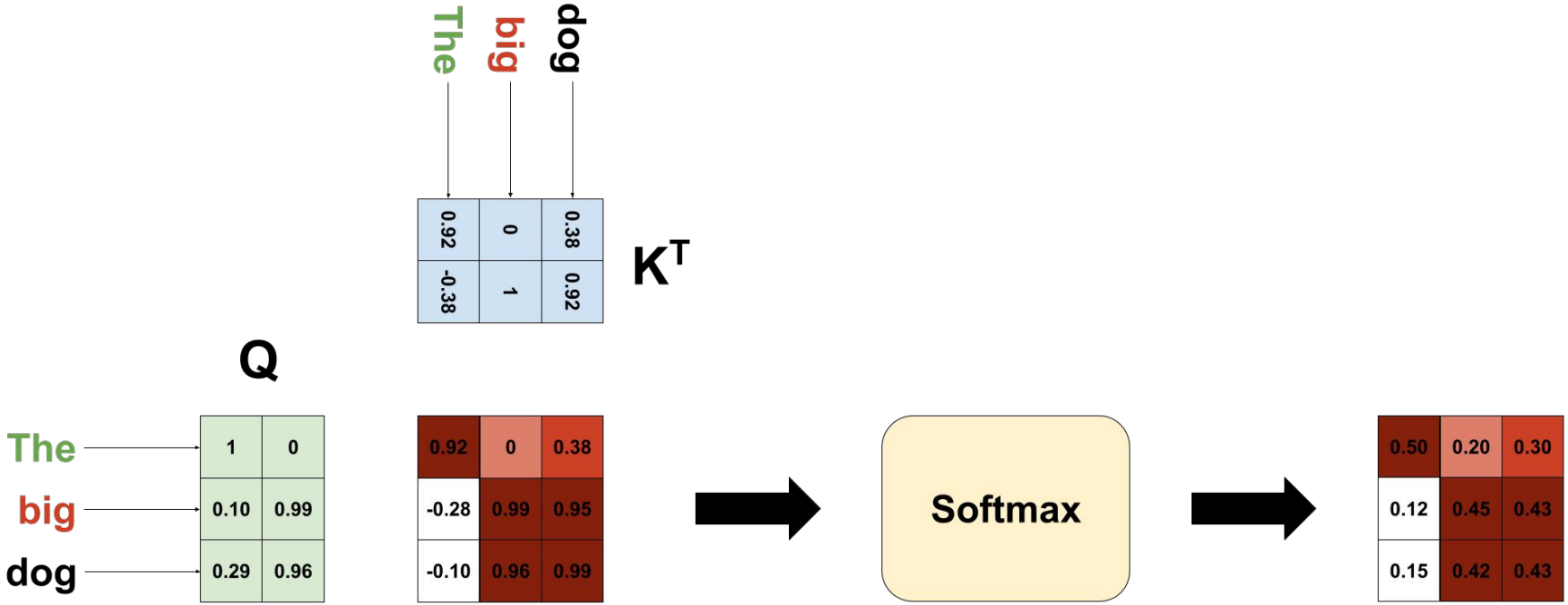
Worked example: “The big dog” (1/3)

The big dog

The : (0.50, -0.87)
big : (-0.70, 0.70)
dog : (0.81, 0.59)

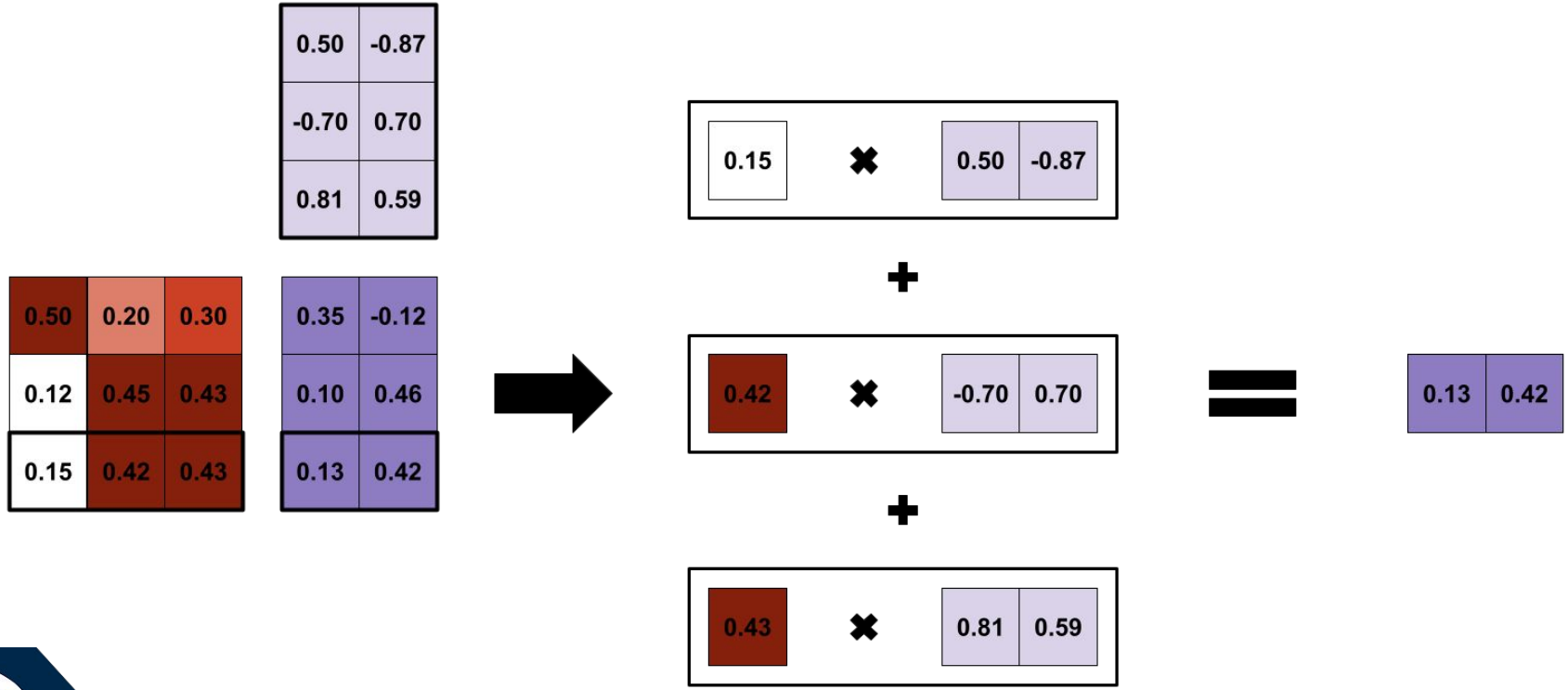


Worked example: “The big dog” (2/3)



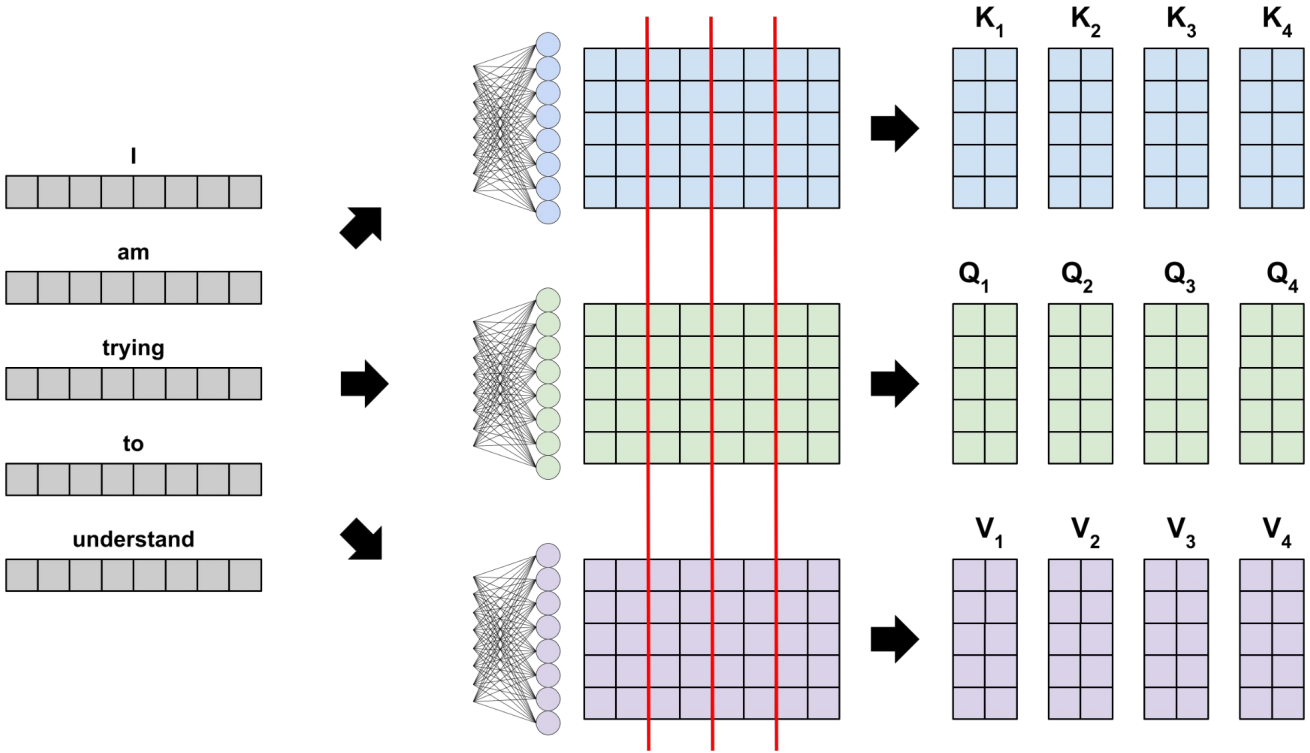
Worked example: “The big dog” (3/3)

Weighted sum — “dog” becomes 0.15 × The + 0.42 × big + 0.43 × dog: its vector now carries “big”.



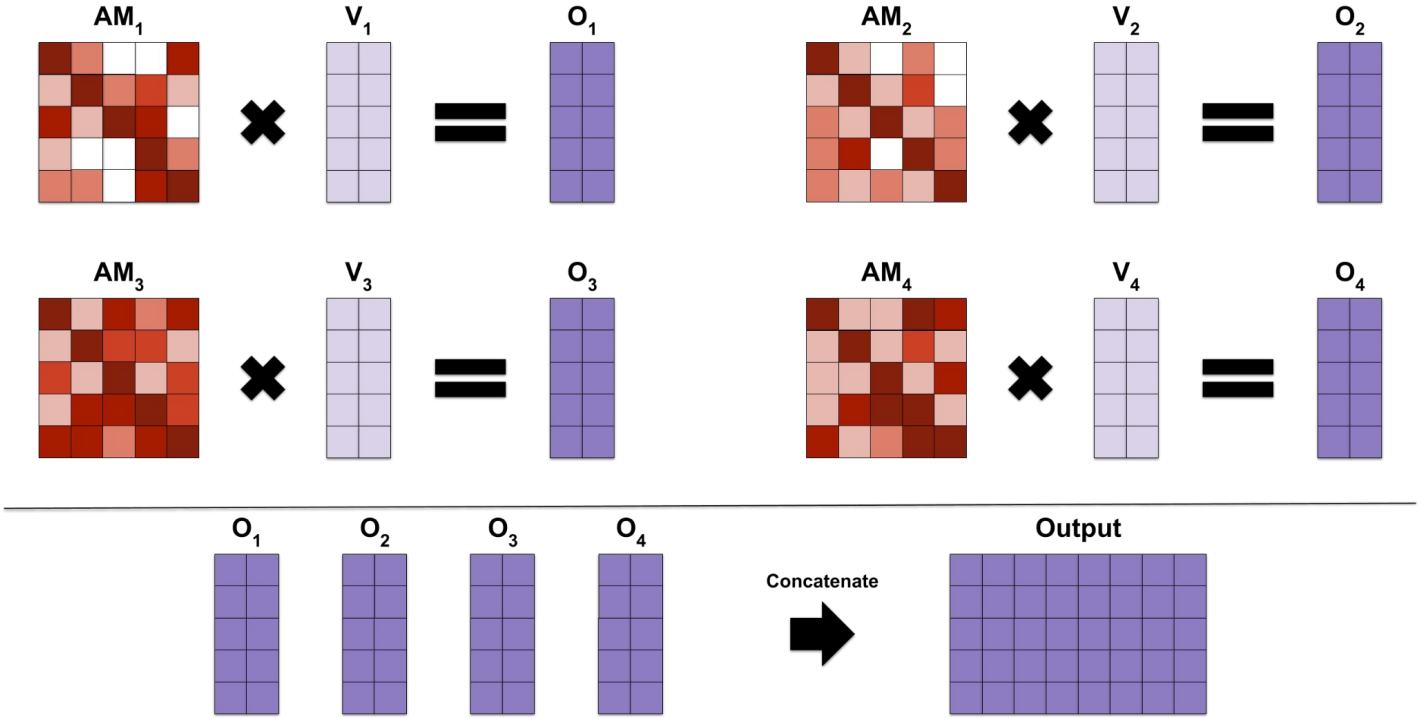
Multi-head attention - 1

Qwen3-8B 32 heads × 128 dimensions = 4,096



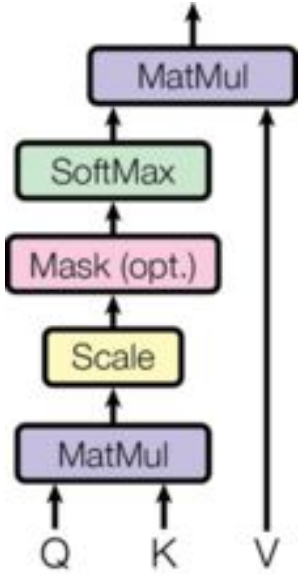
Multi-head attention - 2

Concatenation : the h head outputs are put side by side, then mixed by a final linear layer W_O.

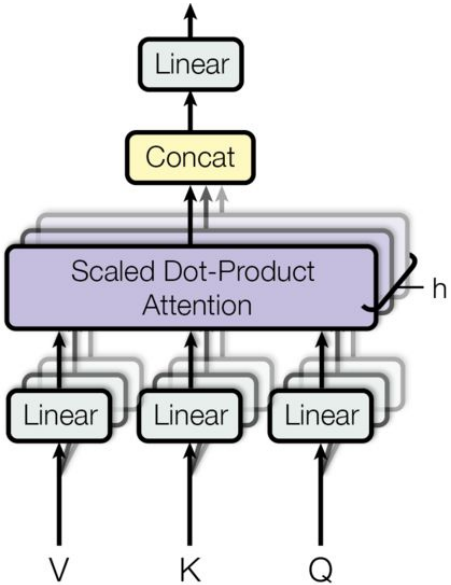


Attention in the original paper

Scaling by 1/√d□ — "for large values of d□, the dot products grow large in magnitude", pushing the softmax into regions with tiny gradients. *(Vaswani et al., 2017)*



Scaled dot-product attention



Multi-head attention

Attention(Q, K, V) = softmax(QK^T / √d□) V

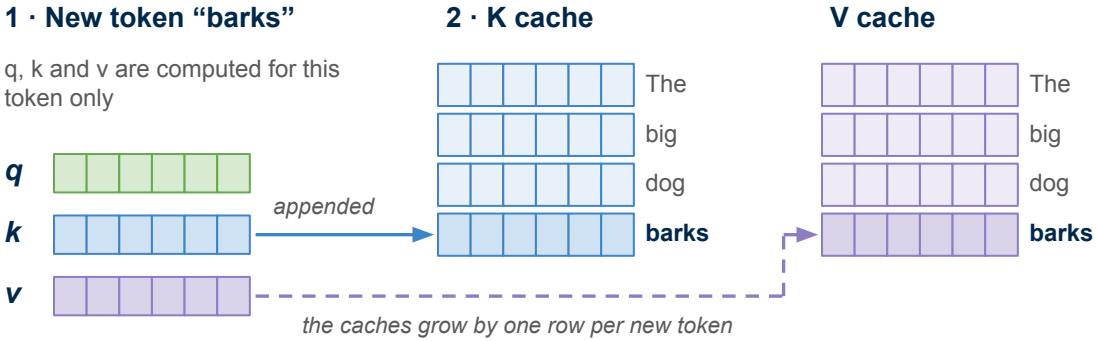
MultiHead = Concat(head$_1$, …, head□) · W^O
with head$_i$ = Attention(Q·W$_i$_Q, K·W$_i$_K, V·W$_i$_V)

Base model: h = 8 heads, d□ = d$_v$ = 64, d_model = 512.

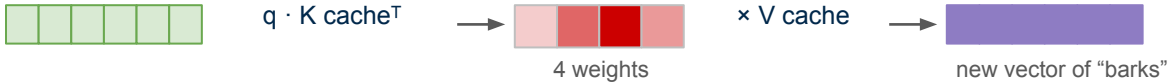
Figure 2 of Vaswani et al., NeurIPS 2017 (arXiv:1706.03762), reproduced with attribution as permitted by the authors.

What is the KV cache?

KV cache : during generation, the keys and values of the tokens already processed are kept in GPU memory, so each new token only computes its own.



3 · Attention of the new token over all cached tokens

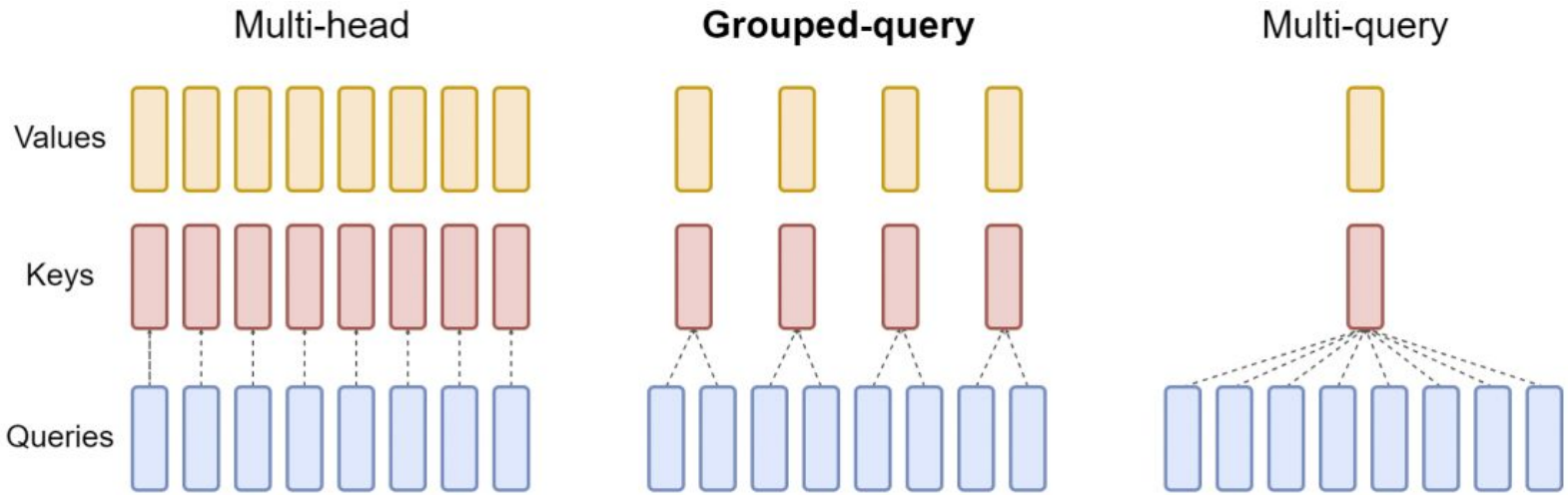


Fewer KV heads = a smaller cache: that is grouped-query attention (next slide).

Without the cache, the keys and values of all previous tokens would be recomputed at every new token.

Sharing keys and values: grouped-query attention

Grouped-query attention (GQA) : several query heads share one key/value head, which shrinks the memory needed to store K and V.

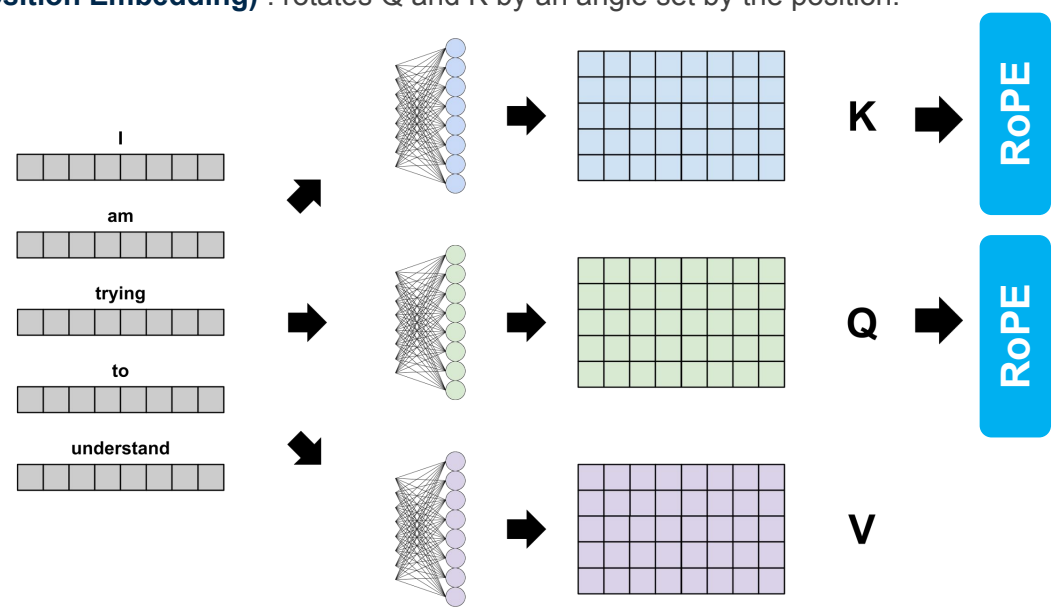


KV cache: during generation, the keys and values of all previous tokens are kept in GPU memory.

Figure 2 of Ainslie et al., EMNLP 2023 github.com/fkodom/grouped-query-attention-pytorch.

Rotary Positional Embeddings (RoPE)

RoPE (Rotary Position Embedding) : rotates Q and K by an angle set by the position.



$$\text{softmax}\left(\frac{\mathbf{Q_2}\mathbf{K_2}^\intercal}{\sqrt{d_K}}\right)\mathbf{V}$$

$$\langle \text{RoPE}\left(q_j, m\right), \text{RoPE}\left(k_j, n\right)\rangle = \text{RoPE}\left(q_j k_j, m-n\right)$$

Qwen3-8B RoPE base θ = 1,000,000

RoPE: rotate queries and keys

Rotation : each pair of coordinates is turned by $m \cdot \theta_i$, so $q \cdot k$ depends only on the distance.

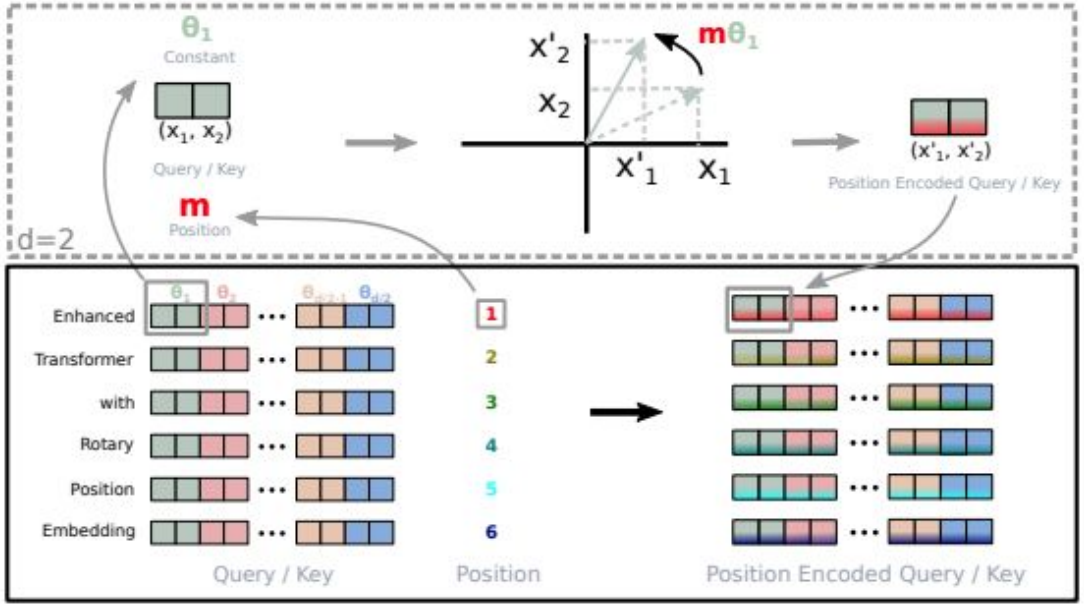


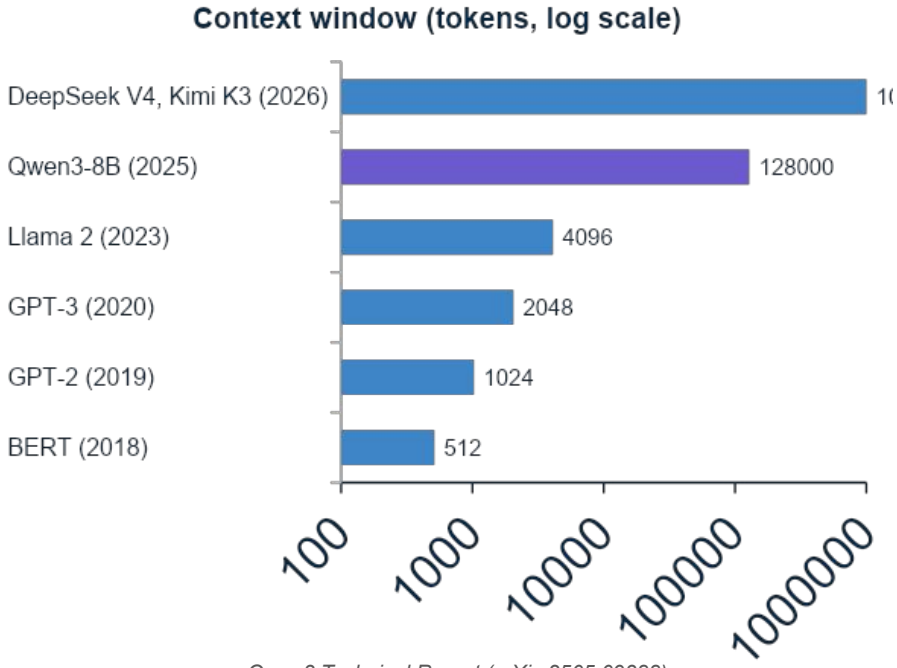
Figure 1 of Su et al., "RoFormer: Enhanced Transformer with Rotary Position Embedding" github.com/lucidrains/rotary-embedding-torch.

Extending the context window

Context window : the maximum number of tokens a model can attend to at once.

- **Beyond the training length,** RoPE produces rotation angles never seen in training, and quality collapses.
- **Fixes:** a larger RoPE base, position interpolation, YaRN (Peng et al., 2023), plus a final training phase on long documents.

Qwen3-8B trained up to 32k tokens (RoPE base $10^4 \rightarrow 10^6$), 128k at inference with YaRN × 4



Qwen3 Technical Report (arXiv:2505.09388).



Transformers architectures

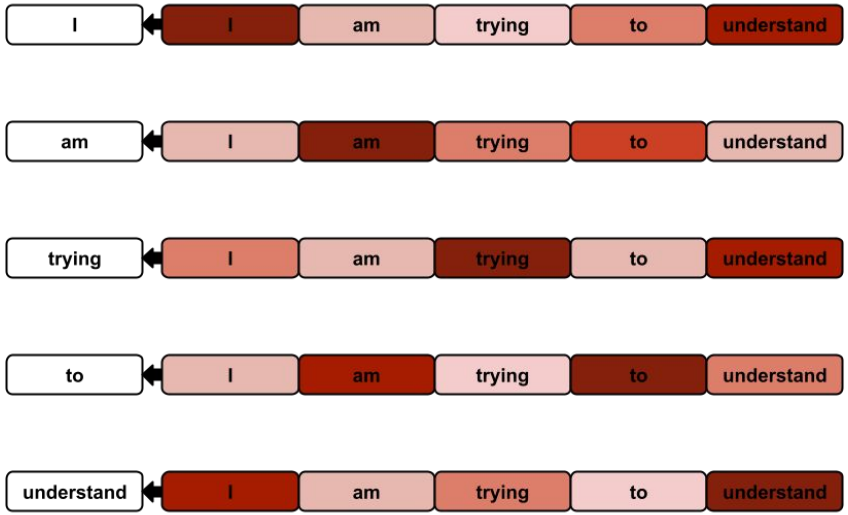
Transformer architectures

Three families : encoder-only (understand), decoder-only (generate), encoder-decoder (turn one sequence into another).



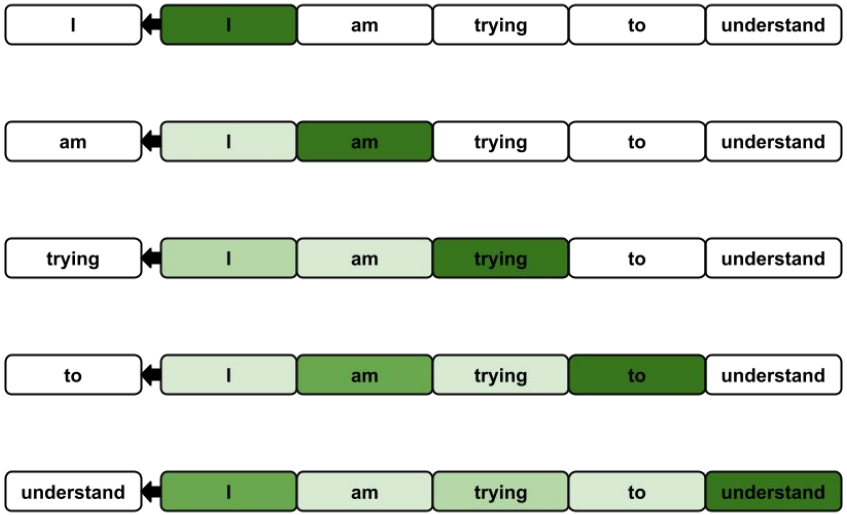
Bidirectional vs Unidirectional attention

Bidirectional attention for Encoder



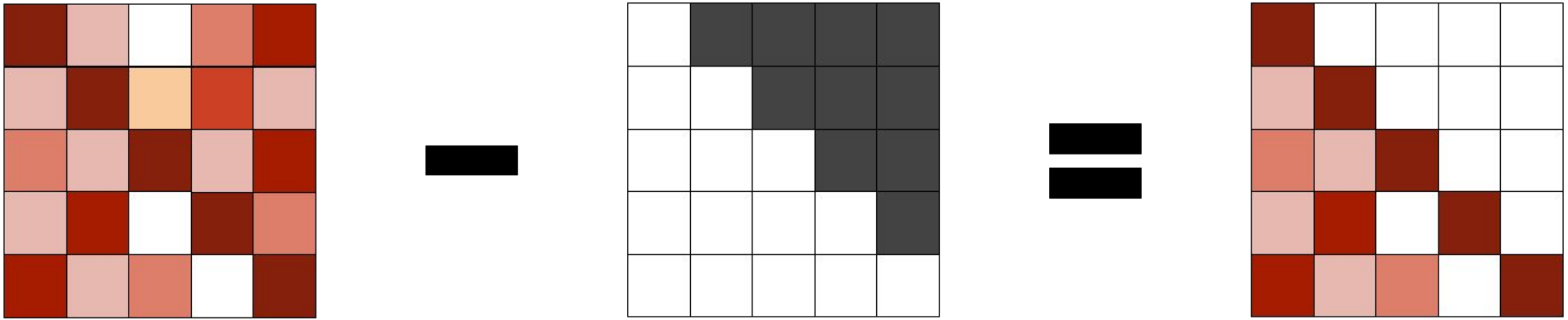
VS

Unidirectional attention for Decoder



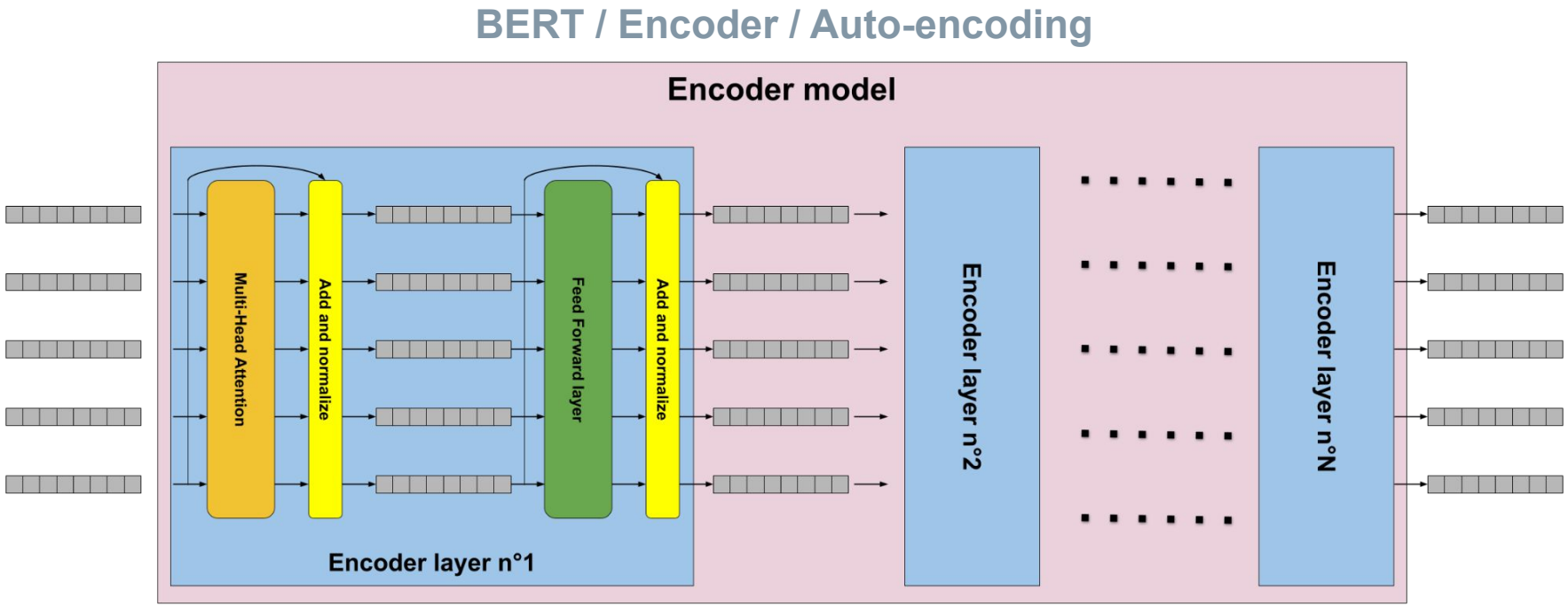
Mask attention

Causal mask : scores above the diagonal are set to $-\infty$ before the softmax, so future tokens get a weight of exactly 0.



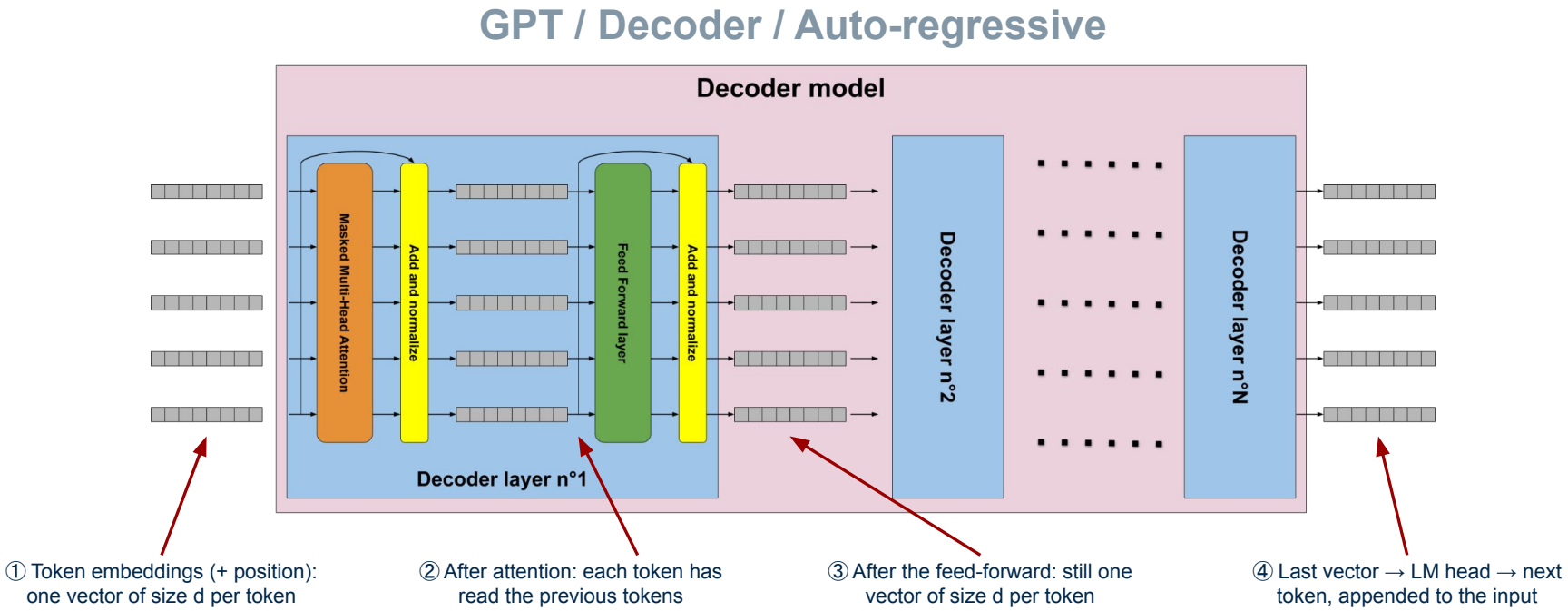
Encoder architecture

Encoder : bidirectional: every token sees the whole sentence. Still used for classification and for search / RAG embeddings.



Decoder architecture

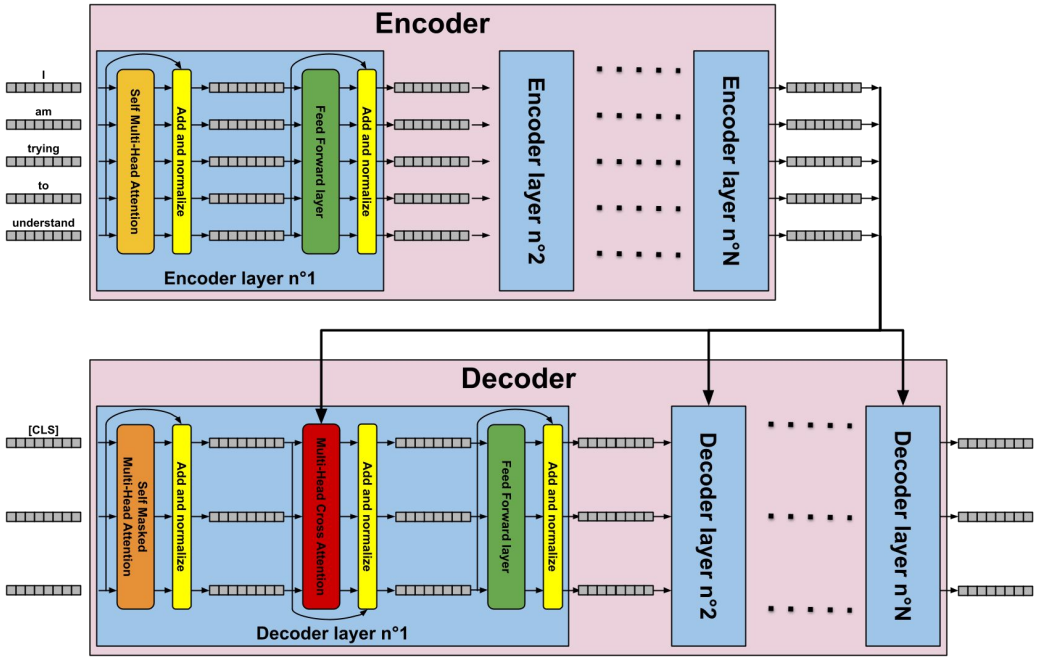
Decoder : causal and autoregressive: it generates one token at a time, each token seeing only the previous ones.



Encoder-Decoder architecture

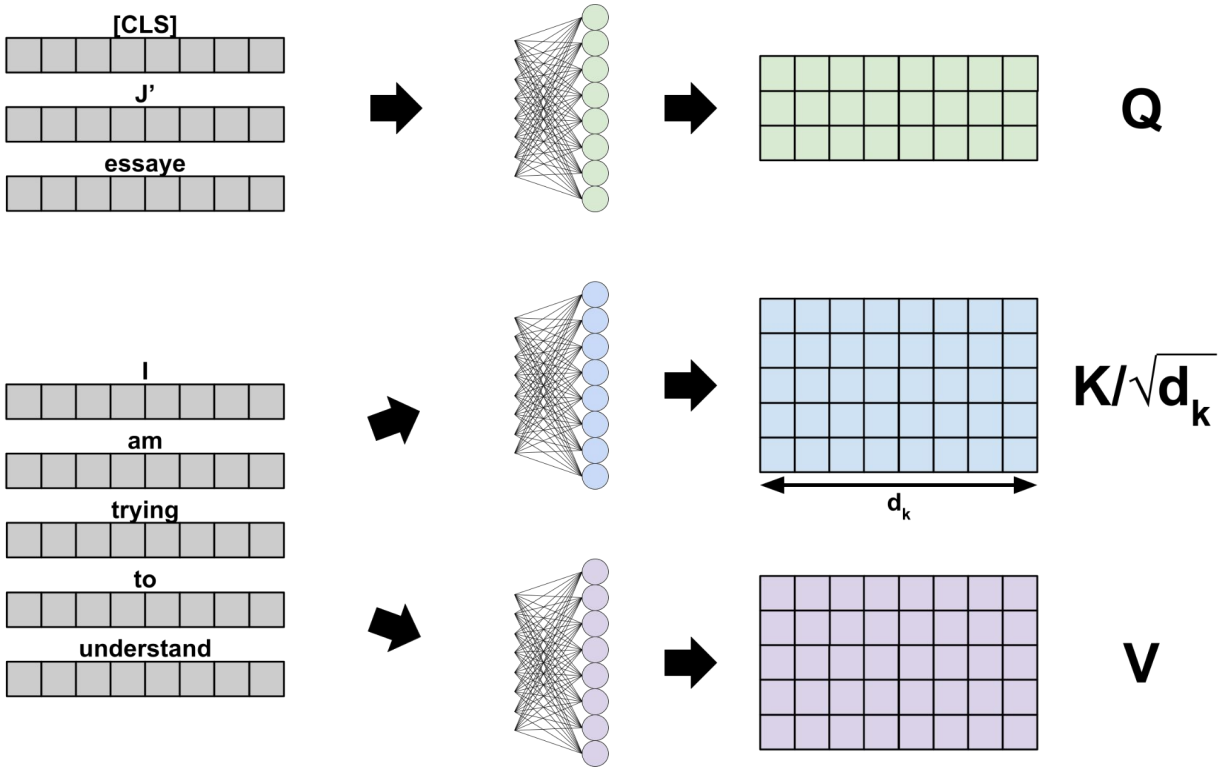
Encoder-decoder : the encoder reads the source; the decoder generates the target and reads the encoder through cross-attention.

T5 / Encoder-Decoder / Sequences-to-Sequences



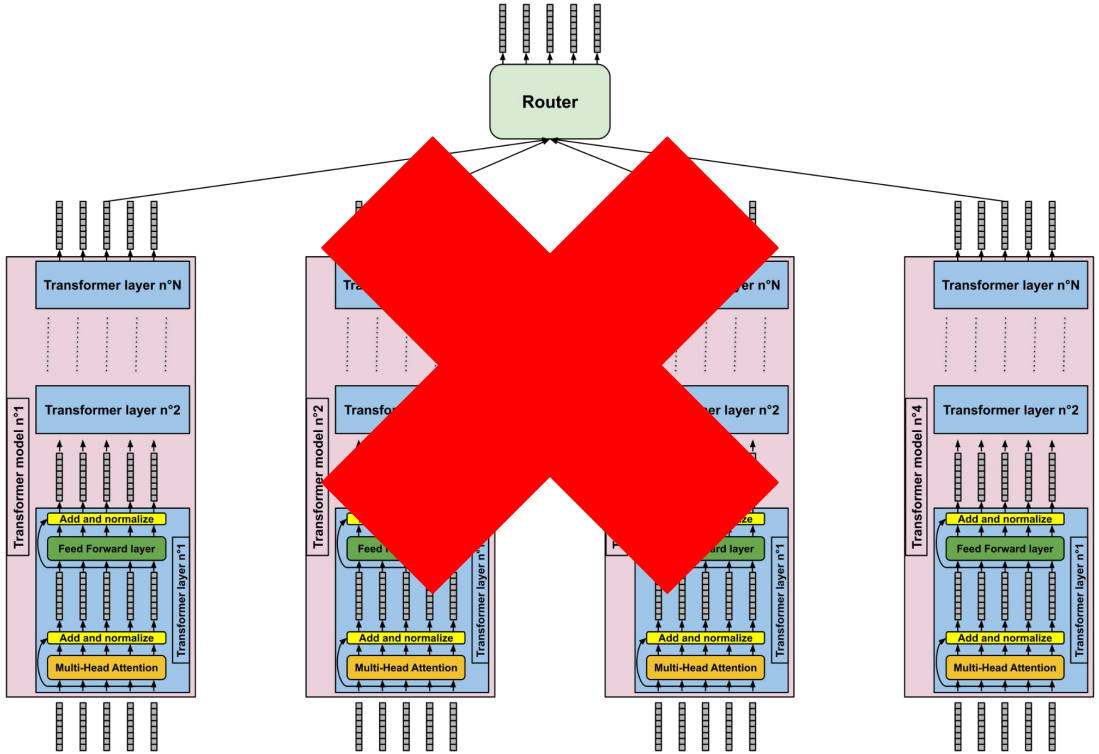
Cross attention

Cross-attention — the queries come from the decoder, the keys and values from the encoder.



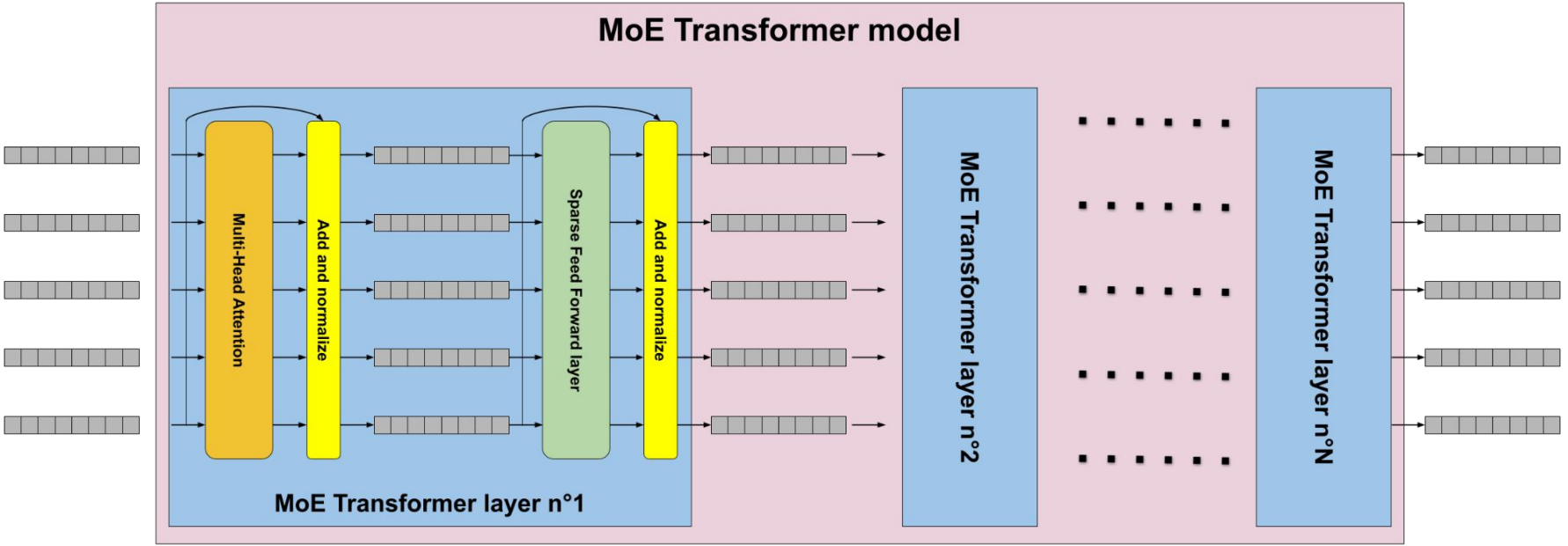
Misunderstanding about Mixture of Experts

Mixture of Experts (MoE) is not several complete models behind a router: the experts live inside each layer.

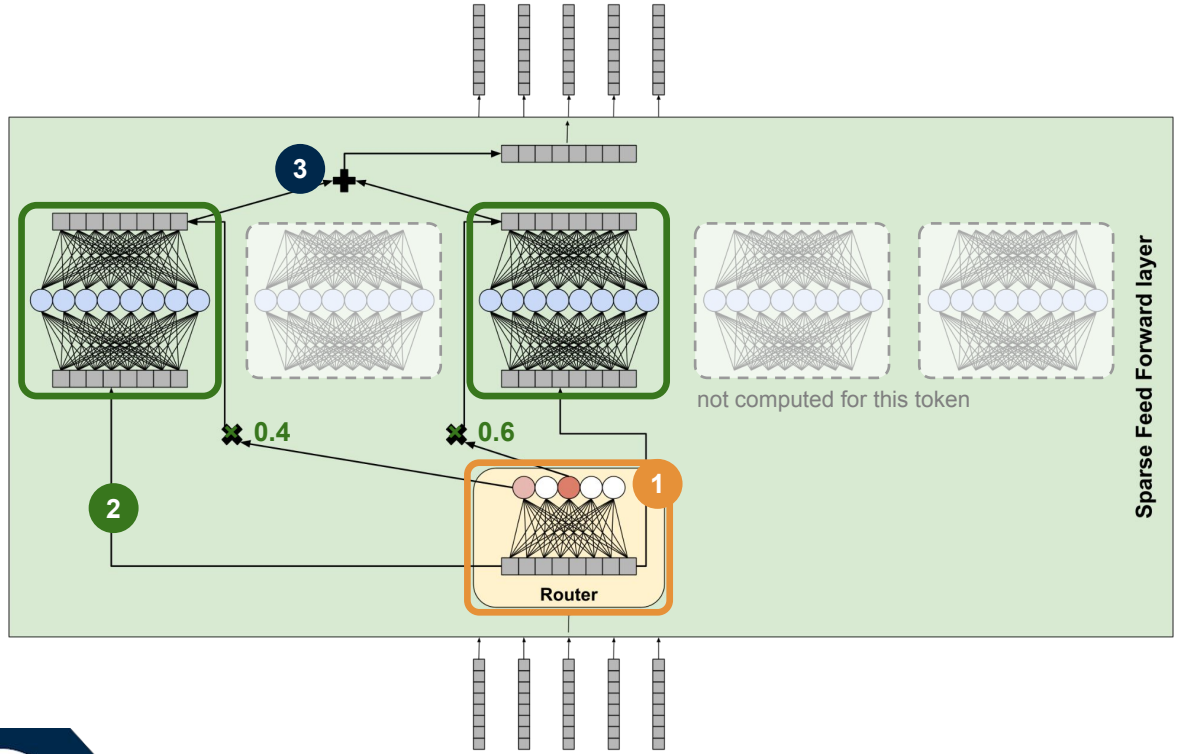


Mixture of Expert Transformer Architecture

MoE layer the feed-forward block of each layer is replaced by a sparse one, attention is unchanged.



Sparse Feed Forward Layer



1. The router scores the 5 experts and keeps the top 2 (darkest circles).
2. The token goes through these 2 experts only (green); the other 3 are skipped (grey).
3. Their outputs are multiplied by the router weights (×) and summed (+).

Each token, in each layer, can pick different experts.

Mixture of Experts in today's open models

Active parameters : the parameters actually used for one token. All the experts must still be loaded in memory.

Model	Release	Total	Active / token	Experts used / total
Mixtral 8x7B	Dec 2023	47 B	13 B	2 / 8
DeepSeek-V3	Dec 2024	671 B	37 B	8 / 256
Qwen3-30B-A3B	Apr 2025	30.5 B	**3.3 B**	8 / 128
gpt-oss-120b	Aug 2025	117 B	5.1 B	4 / 128
Kimi K3	2026	2.8 T	104 B	16 / 896

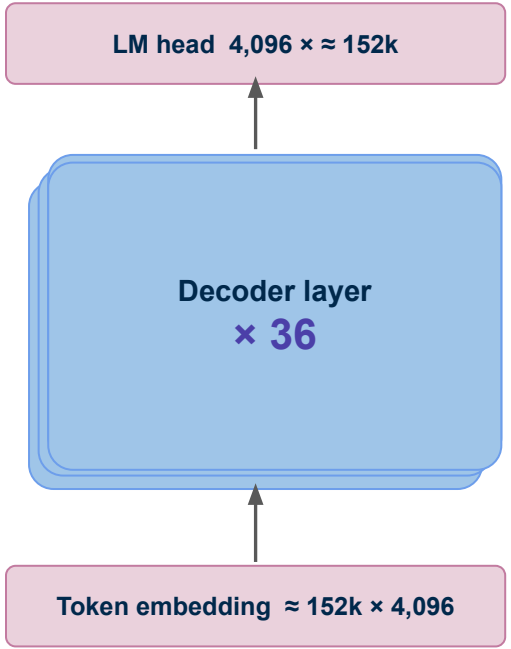
model cards and technical reports (Mixtral, DeepSeek-V3, Qwen3, gpt-oss, Kimi K3 and friends.



LLM today

Qwen3-8B

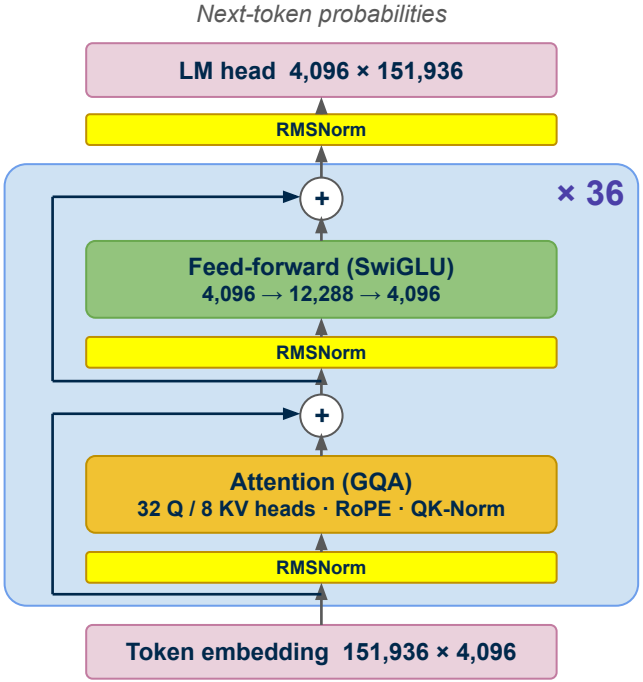
Qwen3-8B	
Type	Dense, decoder-only Transformer
Parameters	8.2 B
Layers	36
Vector size (d)	4,096
Attention heads	32 query / 8 key-value (GQA)
Vocabulary	151,669 tokens (byte-level BPE)
Context	32k tokens
Training data	36 T tokens, 119 languages



From the 2017 Transformer to Qwen3-8B

	Transformer base (2017)	Qwen3-8B (2025)
Architecture	Encoder-decoder	Decoder only
Layers	6 + 6	36
Vector size d	512	4,096
Attention	8 heads (MHA)	32 Q / 8 KV heads (GQA), QK-Norm
Feed-forward	ReLU, 2,048	SwiGLU, 12,288
Normalization	LayerNorm after each sub-layer	RMSNorm before each sub-layer
Position	Sinusoidal, added to the input	RoPE inside attention ($\theta = 10^6$)
Vocabulary	≈ 37k (BPE)	151,669 (byte-level BPE)
Parameters	65 M	8.2 B

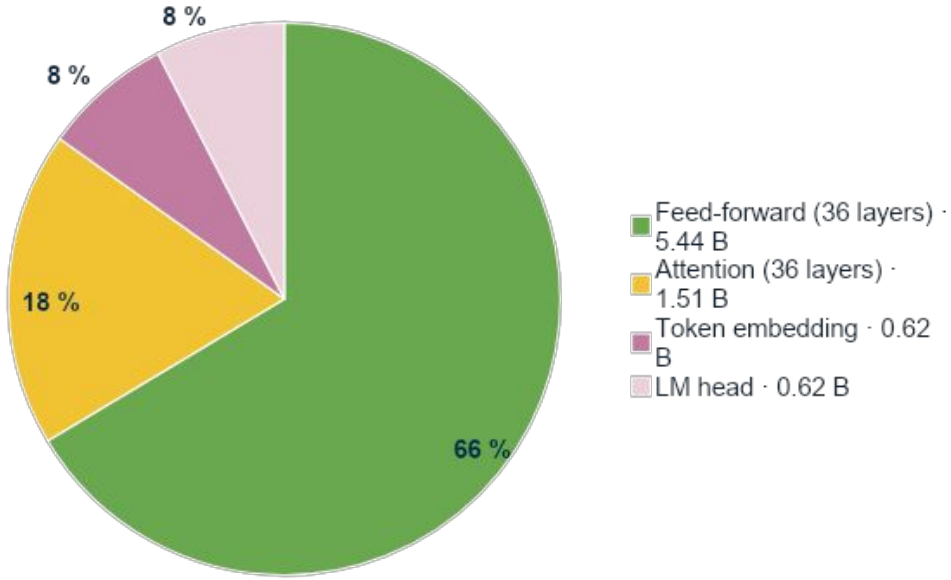
Qwen3 Technical Report (arXiv:2505.09388);n.



Qwen3-8B, drawn from its configuration

8.2 billion parameters ?!

Parameter count : computed from the configuration: d = 4,096, 36 layers, feed-forward 12,288, 32 + 8 heads of 128, vocabulary 151,936.

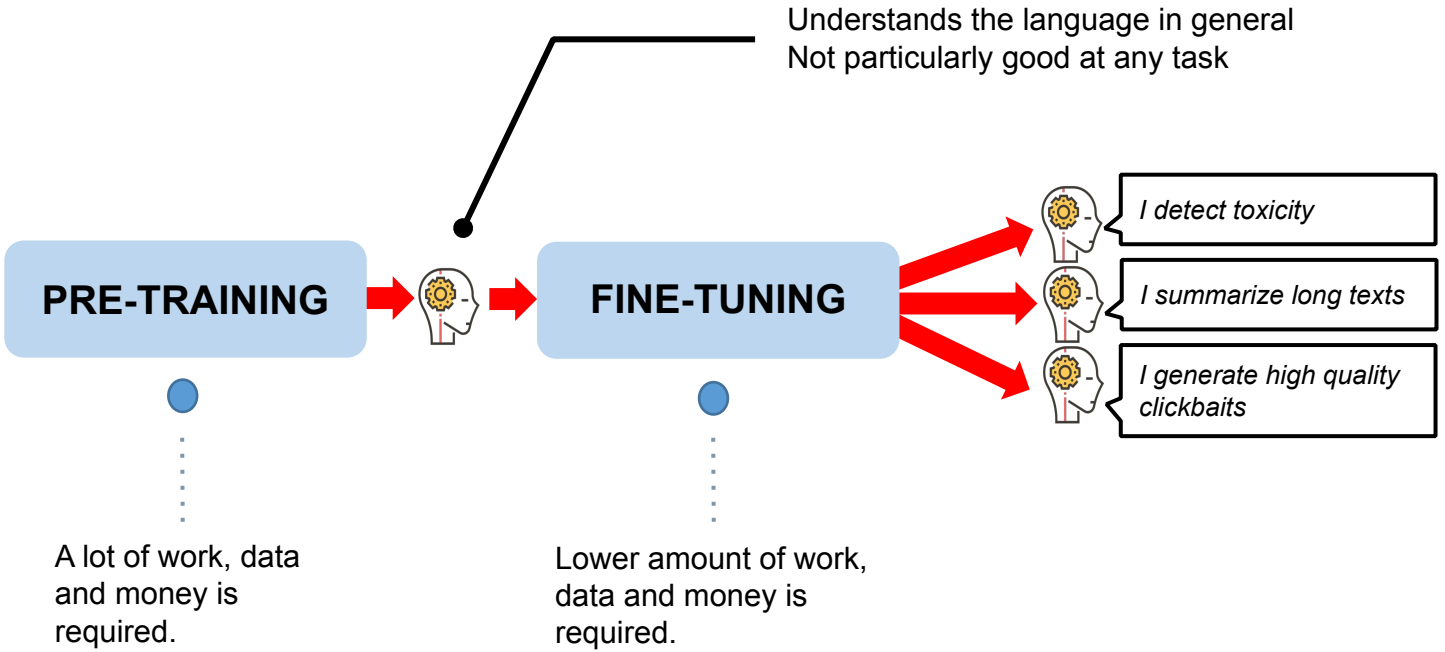


One layer	Shape	Parameters
q_proj	4,096 × 4,096	16.8 M
k_proj	4,096 × 1,024	4.2 M
v_proj	4,096 × 1,024	4.2 M
o_proj	4,096 × 4,096	16.8 M
gate_proj, up_proj	2 × 4,096 × 12,288	100.7 M
down_proj	12,288 × 4,096	50.3 M
Total		**193 M**

Two thirds of the parameters sit in the feed-forward layers.

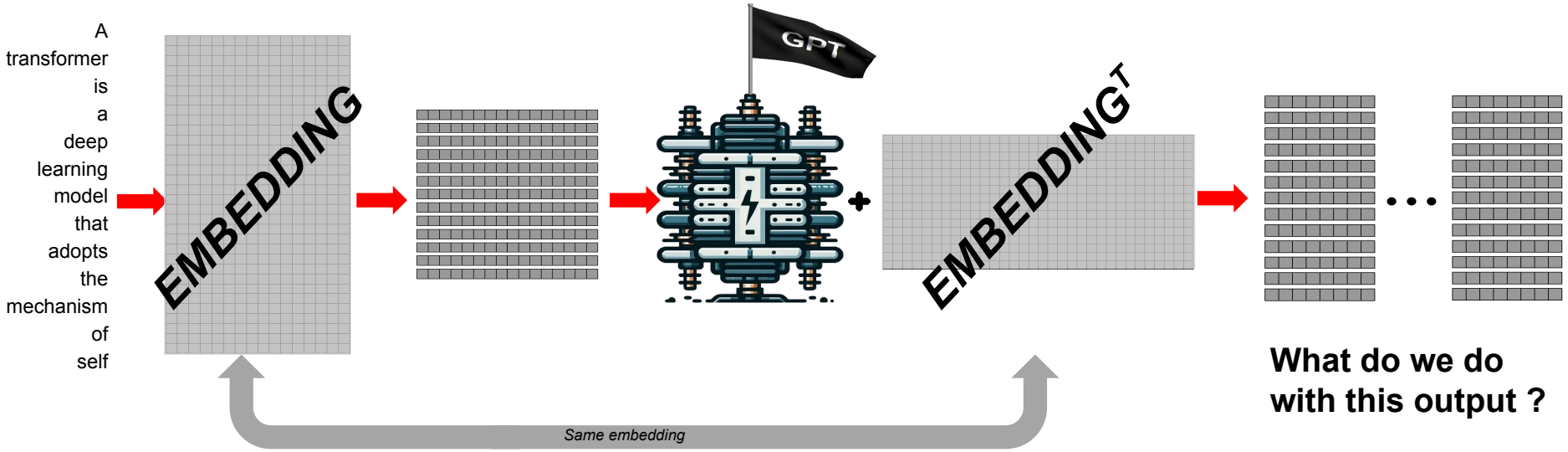
Pre-training, then post-training

Foundation model : pre-trained once on broad data , then adapted to many tasks .



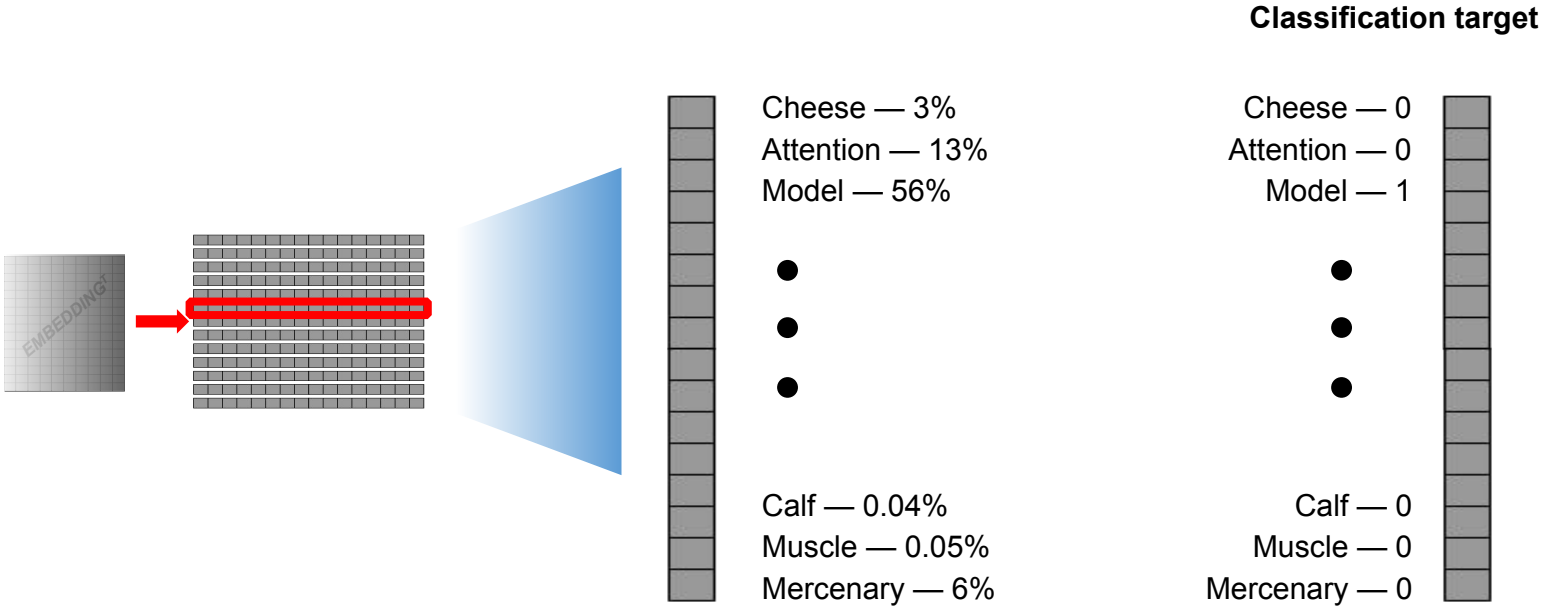
Pre-training a GPT-style model

Self-supervised learning the labels come from the text itself: the target of each position is the next token.



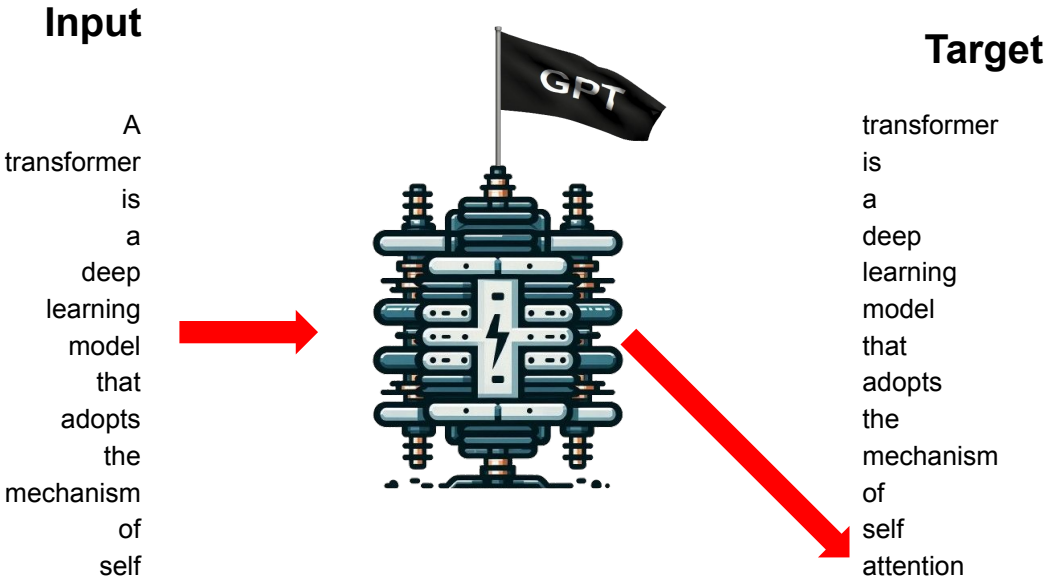
From output vector to next-token probabilities

Cross-entropy loss : −log of the probability given to the correct token



Next-token prediction

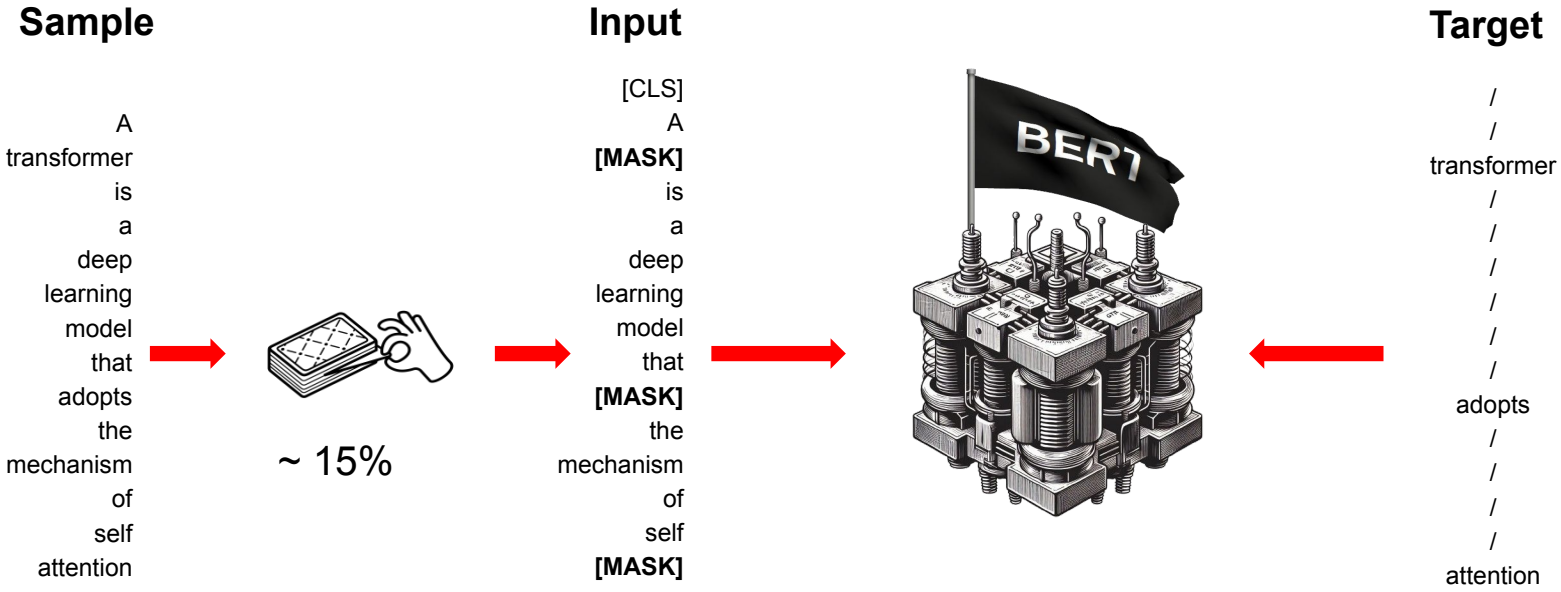
Next-token prediction the target is the input shifted by one, all positions are trained in parallel.



Next word prediction

Pre-training a BERT-style model

Masked language modeling : about 15 % of the tokens are hidden and the model must recover them. *(Devlin et al., 2019)*

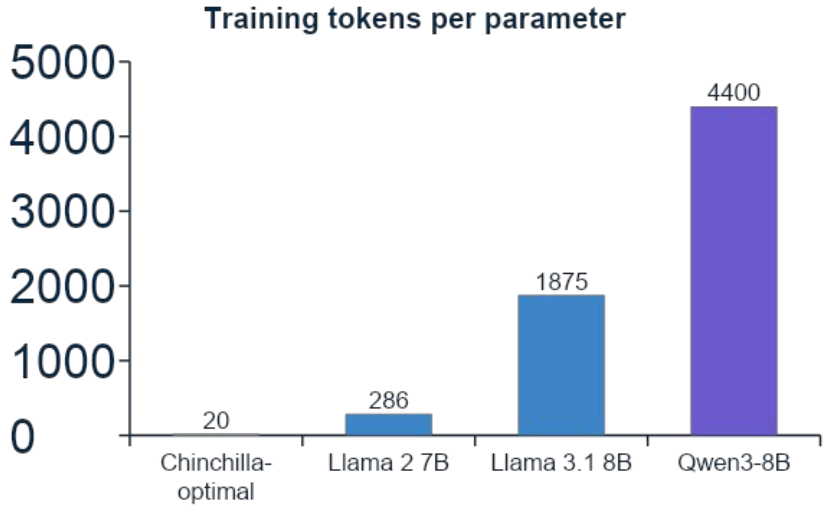


Masked words prediction

Scaling laws

Scaling laws — the loss falls as a power law of model size, data and compute. Training compute ≈ 6 × N parameters × D tokens.
(Kaplan et al., 2020)

FIGURE TO INSERT
Kaplan et al. (2020), “Scaling Laws for Neural Language Models”, Figure 1 (loss vs compute, dataset size, parameters)
https://arxiv.org/abs/2001.08361



Chinchilla rule: ≈ 20 tokens per parameter (Hoffmann et al., 2022). Small models are now trained far beyond it.

Qwen3-8B 36 T tokens: $6 \times 8.2 \cdot 10^{9} \times 36 \cdot 10^{12} \approx 1.8 \cdot 10^{24}$ FLOPs

Pre-training Qwen3: data and phases

1 · General

> 30 T tokens, 4k-token sequences, 119 languages

→ language and world knowledge

2 · Reasoning

≈ 5 T higher-quality tokens: STEM, code, reasoning, synthetic data

→ sharper skills

3 · Long context

hundreds of billions of tokens at 32k, RoPE base raised to 10^6

→ long documents

All Qwen3 models, from 0.6 B to 235 B parameters, see the same ≈ 36 T tokens. **Post-training comes next.**

Qwen3 Technical Report (arXiv:2505.09388), Section 3.

How much GPU memory for Qwen3-8B?

Rule of thumb — memory ≈ bytes per parameter × number of parameters, plus activations and KV cache.

Use	Bytes / parameter	Qwen3-8B (8.2 B)
Inference, BF16	2	≈ 16 GB
Inference, 8-bit	1	≈ 8 GB
Inference, 4-bit	≈ 0.5	≈ 5 GB
Full fine-tuning (Adam, mixed precision)	≈ 16	≈ 130 GB
LoRA on the frozen BF16 model	≈ 2 + adapters	≈ 17 GB
QLoRA on the frozen 4-bit model	≈ 0.5 + adapters	≈ 6 GB

16 bytes = weights 2 + gradients 2 + FP32 copy 4 + Adam moments 4 + 4. **Reference: one H100 GPU = 80 GB.**

Downloading and running Qwen3-8B

Hugging Face Hub — the main platform to download open-weight models, with their tokenizer and configuration.

1 Find the model card

huggingface.co/Qwen/Qwen3-8B

2 Download the weights

≈ 16 GB in BF16

3 Load and generate

the chat template formats the conversation

```
$ pip install -U transformers huggingface_hub
$ hf download Qwen/Qwen3-8B --local-dir $WORK/models/qwen3-8b
```

```
from transformers import AutoTokenizer, AutoModelForCausalLM

path = "Qwen/Qwen3-8B"            # or the local folder
tokenizer = AutoTokenizer.from_pretrained(path)
model = AutoModelForCausalLM.from_pretrained(path, torch_dtype="auto", device_map="auto")

messages = [{"role": "user", "content": "Explain attention in one sentence."}]
text = tokenizer.apply_chat_template(messages, tokenize=False,
        add_generation_prompt=True, enable_thinking=False)
inputs = tokenizer([text], return_tensors="pt").to(model.device)

out = model.generate(**inputs, max_new_tokens=128)
print(tokenizer.decode(out[0][inputs.input_ids.shape[1]:], skip_special_tokens=True))
```

Adapted from the official examples (github.com/QwenLM/Qwen3). enable_thinking=False turns the reasoning mode off.



Post-training

Raw foundation models

Base model : the raw pre-trained model: very capable, but it only continues text.



Very powerful
Huge potential

Hard to control
Poor performance (on specific tasks)

Foundation model = Chaotic monster

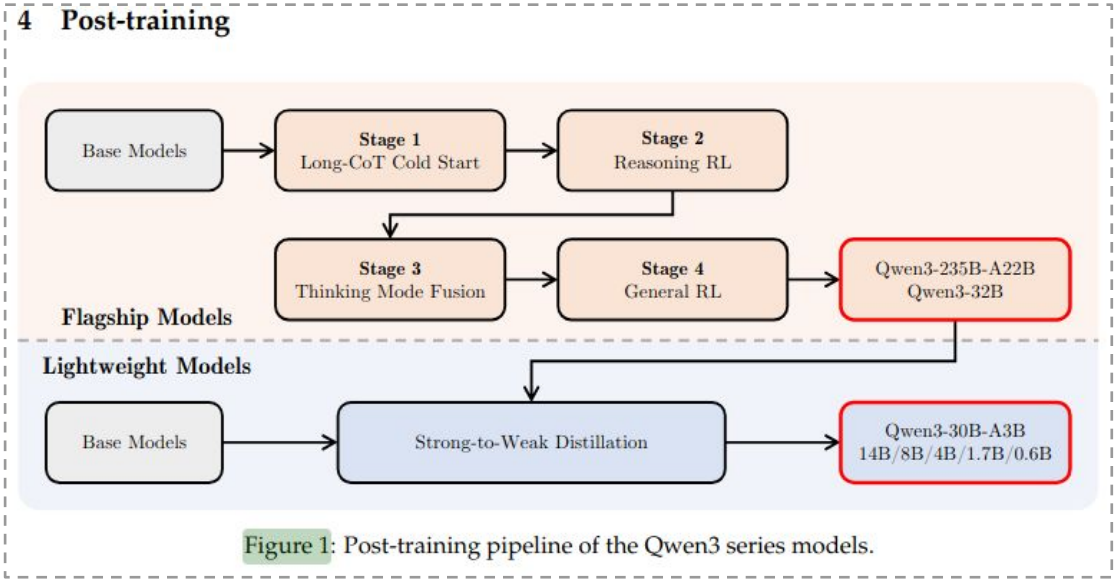
Example of chaos

Instruction following — not learned by pre-training alone: the base model continues the review instead of rating it.



Post-training: from base model to assistant

Post-training : fine-tuning on instructions (SFT), then preference alignment and reinforcement learning.



Flagship models, 4 stages

- Long chain-of-thought SFT
- Reasoning RL (GRPO)
- Thinking-mode fusion
- General RL

Qwen3 Technical Report, Figure 1 https://arxiv.org/abs/2505.09388

Alignment: RLHF and DPO

Alignment : making the model's answers match human preferences: helpful, honest and harmless.

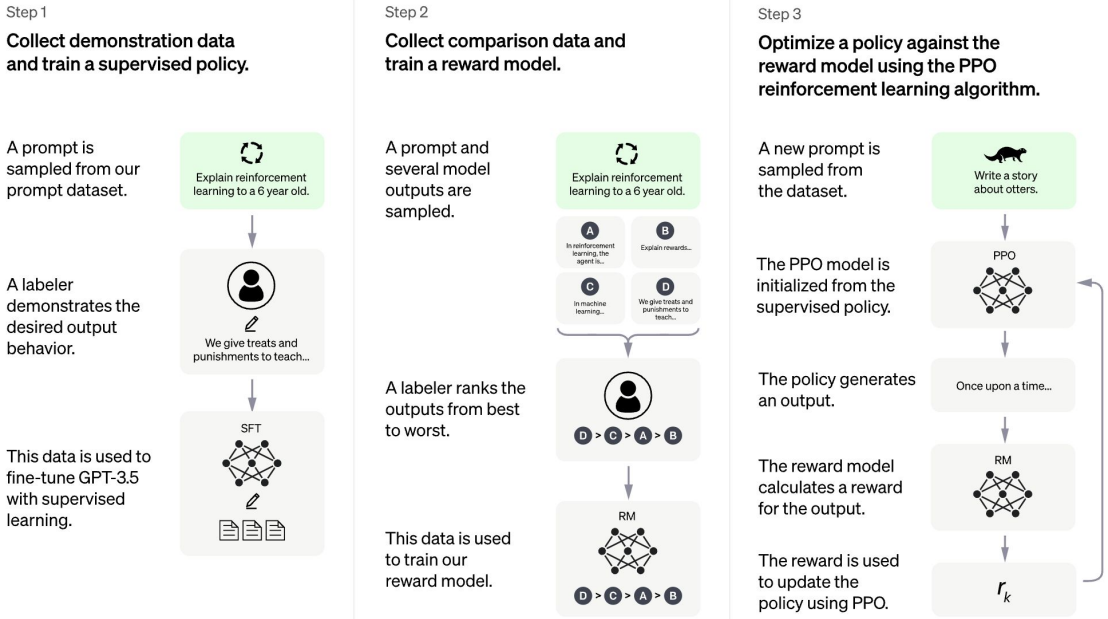


Figure: OpenAI (2022), RLHF pipeline of InstructGPT / ChatGPT (Ouyang et al., arXiv:2203.02155), via github.com/opendilab/awesome-RLHF.

RLHF: the three steps on the left (Ouyang et al., 2022).

DPO (Rafailov et al., 2023): learns directly from pairs of preferred / rejected answers. No reward model, no RL.

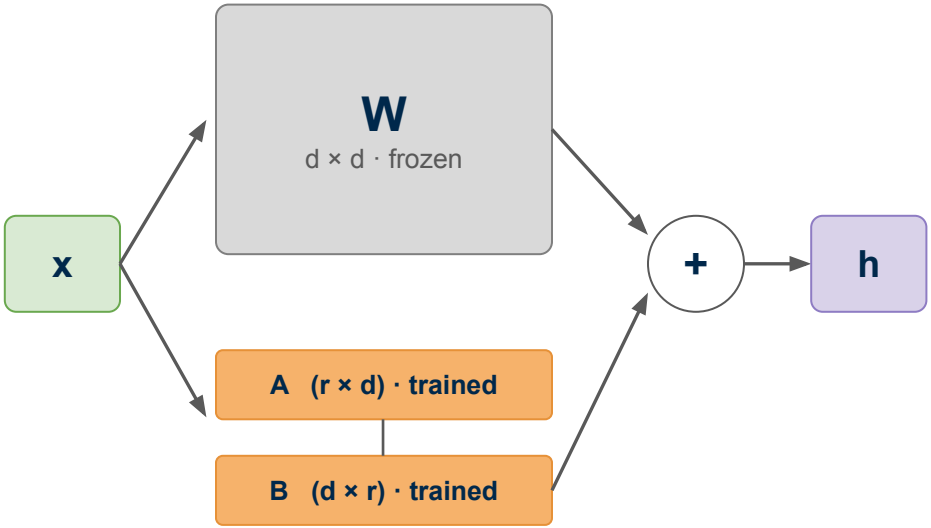
RL with verifiable rewards: maths and code answers checked automatically. This is how reasoning models are trained (DeepSeek-R1, Qwen3).

Prompt engineering, RAG or fine-tuning ?

Your need	Try first
A tone, a format, a behaviour	A system prompt and a few examples
Step-by-step reasoning	A thinking model (e.g. Qwen3 thinking mode)
Answers from your own documents	RAG: retrieve passages, add them to the prompt
A new skill or style, with many examples	LoRA / QLoRA fine-tuning
Answers that match your preferences	DPO on pairs of answers

LoRA in practice

LoRA — freeze the weight matrix W and learn a low-rank update B·A, with a rank r much smaller than d. *(Hu et al., 2021)*



- Only A and B are trained: gradients and optimizer states become tiny.
- QLoRA: the frozen model is stored in 4-bit, so an 8 B model fits on a single GPU.
- After training, B·A can be merged into W: no extra cost at inference.

Qwen3-8B q_proj: d = 4,096, r = 16 → 131 k trained parameters vs 16.8 M frozen (0.8 %)

Diagram after Hu et al., 2021, Figure 1 (arXiv:2106.09685). QLoRA: Dettmers et al., 2023 (arXiv:2305.14314).

